



Universidad de Córdoba

Escuela Politécnica Superior De Córdoba

Grado en ingeniería informática

Ingeniería Sistemas Software Basados en Conocimiento

CUADERNO DE PRÁCTICAS

Realizado por:

Manuel Peinado Cuenca

Curso 24/25

Índice

1. Practica 1	1
1.1. Entorno de programación	1
1.2. First programs	2
1.3. Menus and toolbars	5
1.4. Funciones Importantes	9
1.5. Ejemplo propio	11
2. Practica 2	12
2.1. Layout management	12
2.2. Events and signals	14
2.3. Funciones Importantes	17
2.4. Editor de texto	18
2.5. Aplicación básica	20
3. Practica 3	21
3.1. Dialogs	21
3.2. Widgets	24
3.3. Funciones Importantes	31
3.4. Editor de texto MVC	32
3.4.1. ckAppEditorTexto	32
3.4.2. ckVtsEditorTexto	32
3.4.3. ckModAppEditorTexto	34
3.4.4. ckCtrlEditorTexto	34
3.5. Calculadora MVC	35
3.5.1. ckAppCalculadora	35
3.5.2. ckVtsCalculadora	35
3.5.3. ckModAppCalculadora	36
3.5.4. ckCtrlCalculadora	37
4. Parcial 1	38
4.1. Código desarrollado	38
4.1.1. Vista	38
4.1.2. Becas	44
4.1.3. Puesto de trabajo	47
4.1.4. Imagen de la vista creada	50
4.2. Video explicativo	51
5. Practica 4	52
5.1. Resumen de mi experiencia realizando el tutorial	52
5.2. Lenguajes de marcas	53
5.2.1. RDF	53
5.2.2. OWL	53
5.2.3. Turtle	53
6. Practica 5	54
7. Parcial 2	55
7.1. Archivos OWL	55
7.1.1. Esq-dom	55
7.1.2. Becas	57
7.1.3. Puesto_trabajo	63

7.2. Vista de la aplicación 71

7.3. Video explicativo 76

1. Practica 1

1.1. Entorno de programación

En este apartado me he adaptado al entorno de programación de "Spyder" ya que es la primera vez que utilizo este entorno de programación.

Como podemos ver en la figura 1 a la izquierda tenemos una barra lateral donde se ve el proyecto que tenemos abierto con los directorios y los ficheros que tiene dentro, además en el centro de la pantalla vemos el código que tenemos abierto. Por último, en la parte de abajo a la derecha tenemos una terminal.

Además, si queremos ejecutar nuestro código podemos hacer click en la flecha verde que se encuentra debajo de la opción "Ejecutar" y si quisiéramos crear un proyecto tendríamos que irnos a "Proyectos" donde tendríamos la opción de crear un nuevo proyecto.

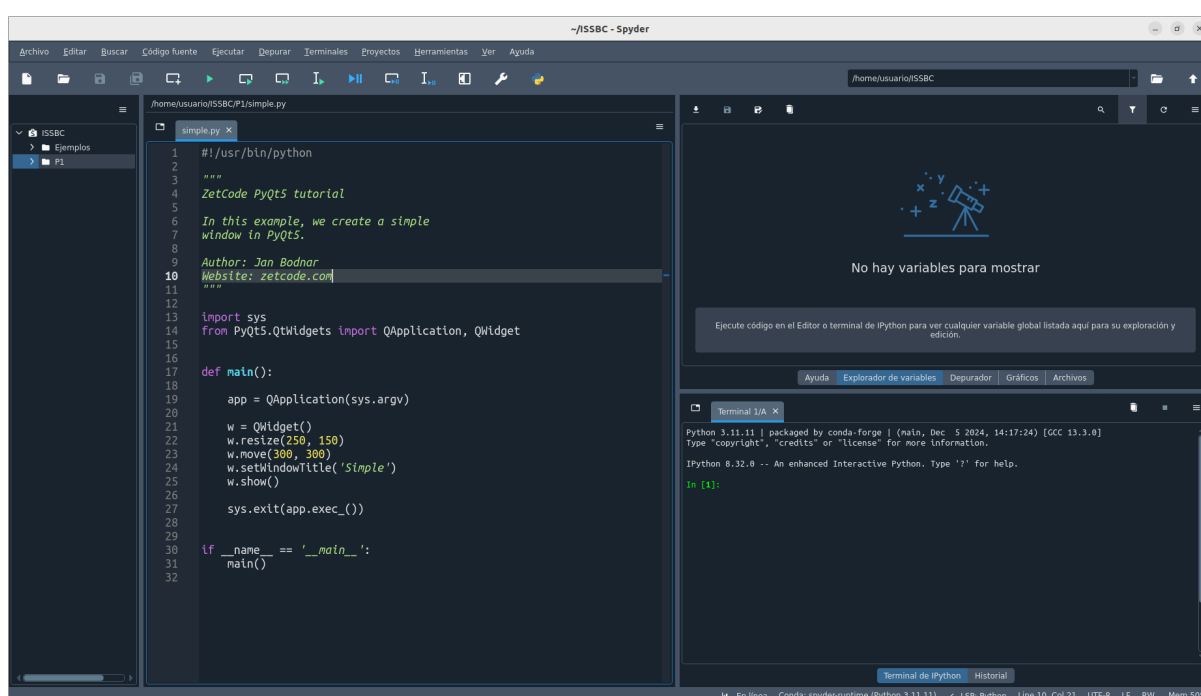


Figura 1: Entorno de programación Spyder

1.2. First programs

En este apartado del tutorial nos introduce la librería "PyQt5" de Python, y a su vez nos enseña algunas de las funciones mas básicas de la misma, entre ellas encontramos las siguientes:

En el ejemplo "simple.py" nos muestra como crear una ventana y como indicar el tamaño de la misma y la posición de la pantalla en la que va a aparecer.

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget

def main():

    app = QApplication(sys.argv)

    w = QWidget()
    w.resize(250, 150)
    w.move(300, 300)
    w.setWindowTitle('Simple')
    w.show()

    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

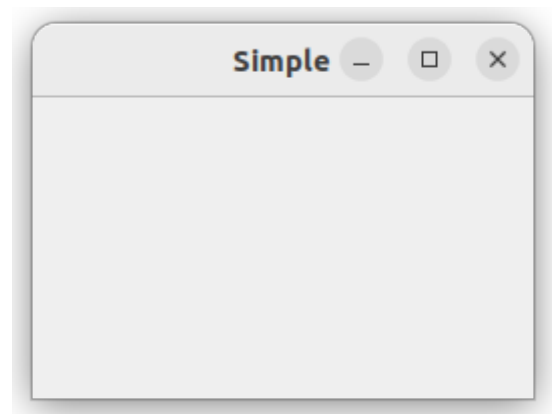


Figura 2: [Ejemplo simple.py](#)

En el ejemplo "icon.py" nos enseña como podemos añadir un icono a nuestra ventana para que identifiquemos mas facilmente que es nuestra aplicacion la que se esta ejecutando

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget
from PyQt5.QtGui import QIcon

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('Icon')
        self.setWindowIcon(QIcon('web.png'))
        self.show()

def main():

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```



Figura 3: [Ejemplo icon.py](#)

En el ejemplo "tooltip.py" nos explica como añadirle a nuestras ventanas un cuadro de texto flotante con alguna información sobre el objeto que tiene el ratón encima.

```
import sys
from PyQt5.QtWidgets import QWidget, QToolTip, QPushButton, QApplication
from PyQt5.QtGui import QFont

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        QToolTip.setFont(QFont('SansSerif',10))

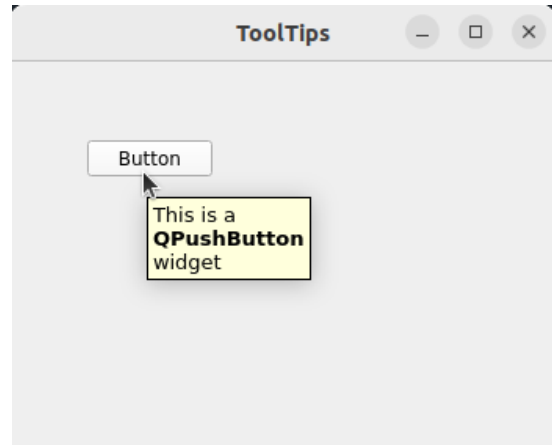
        self.setToolTip('This is a <b>QWidget</b> widget')
        btn = QPushButton('Button', self)
        btn.setToolTip('This is a <b>QPushButton</b> widget')
        btn.resize(btn.sizeHint())
        btn.move(50, 50)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('ToolTips')
        self.show()

def main():

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 4: [Ejemplo tooltip.py](#)

En el ejemplo "quit.button.py" nos explica como crear una ventana mediante una clase lo cual es muy útil, además de explicarnos como crear un botón y como hacer que al pulsar dicho botón se cierre la ventana.

```
import sys
from PyQt5.QtWidgets import QWidget, QPushButton, QApplication

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

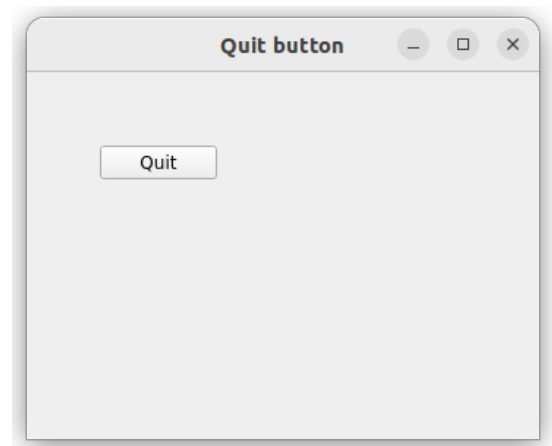
        qbtn = QPushButton('Quit', self)
        qbtn.clicked.connect(QApplication.instance().quit)
        qbtn.resize(qbtn.sizeHint())
        qbtn.move(50, 50)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('Quit button')
        self.show()

def main():

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 5: [Ejemplo quit.button.py](#)

En el ejemplo "messagebox.py" nos explica como crear una nueva ventana de confirmación cuando queramos salir de un archivo.

```
import sys
from PyQt5.QtWidgets import QWidget, QMessageBox, QApplication

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        self.setGeometry(300, 300, 250, 150)
        self.setWindowTitle('Message box')
        self.show()

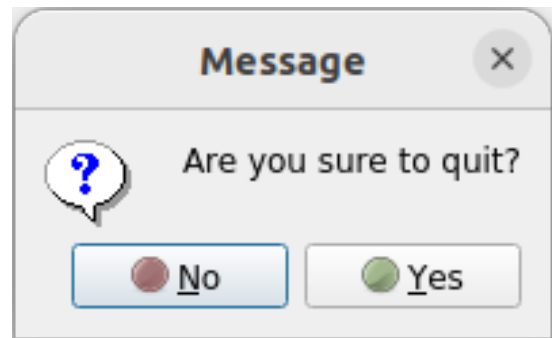
    def closeEvent(self, event):

        reply = QMessageBox.question(self, 'Message',
                                    "Are you sure to quit?", QMessageBox.Yes |
                                    QMessageBox.No, QMessageBox.No)

        if reply == QMessageBox.Yes:
            event.accept()
        else:
            event.ignore()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 6: [Ejemplo messagebox.py](#)

En el ejemplo "center.py" nos muestra como centrar la ventana que creemos sin posiciones absolutas, lo cual nos serviría para cualquier dispositivo sin importar el tamaño de la pantalla.

```
import sys
from PyQt5.QtWidgets import QWidget, QDesktopWidget, QApplication

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        self.resize(250, 150)
        self.center()

        self.setWindowTitle('Center')
        self.show()

    def center(self):

        qr = self.frameGeometry()
        cp = QDesktopWidget().availableGeometry().center()
        qr.moveCenter(cp)
        self.move(qr.topLeft())

def main():

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 7: [Ejemplo center.py](#)

1.3. Menus and toolbars

En este apartado del tutorial nos explica como crear barras de estado, barras de herramientas para añadir distinta información a nuestra ventana y para permitir desempeñar diferentes acciones en la misma:

En el ejemplo "statusbar.py" nos enseña como podemos crear una barra de estados para mostrar cierta información en la misma cuando sea necesario.

```
import sys
from PyQt5.QtWidgets import QMainWindow, QApplication

class Example(QMainWindow):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.statusBar().showMessage('Ready')

        self.setGeometry(300, 300, 250, 150)
        self.setWindowTitle('Statusbar')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

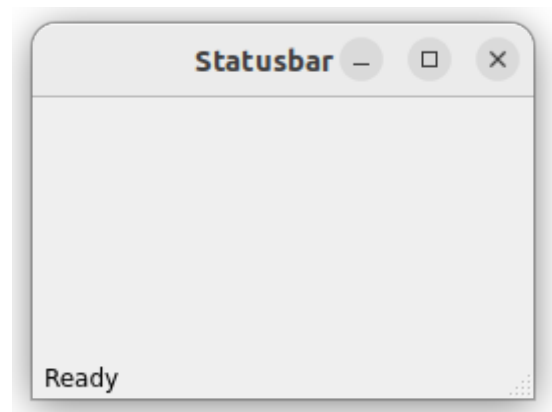


Figura 8: [Ejemplo statusbar.py](#)

En el ejemplo "simple.menu.py" nos muestra como crear una barra de herramientas con la opción de Salir y a su vez cuando pongamos el ratón encima de la misma nos mostrará en la barra de estados que ese botón nos saca de la aplicación.

```
import sys
from PyQt5.QtWidgets import QMainWindow, QAction, qApp, QApplication
from PyQt5.QtGui import QIcon

class Example(QMainWindow):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        exitAct = QAction(QIcon('exit.png'), '&Exit', self)
        exitAct.setShortcut('Ctrl+Q')
        exitAct.setStatusTip('Exit application')
        exitAct.triggered.connect(qApp.quit)

        self.statusBar()

        menubar = self.menuBar()
        fileMenu = menubar.addMenu('&File')
        fileMenu.addAction(exitAct)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Simple menu')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

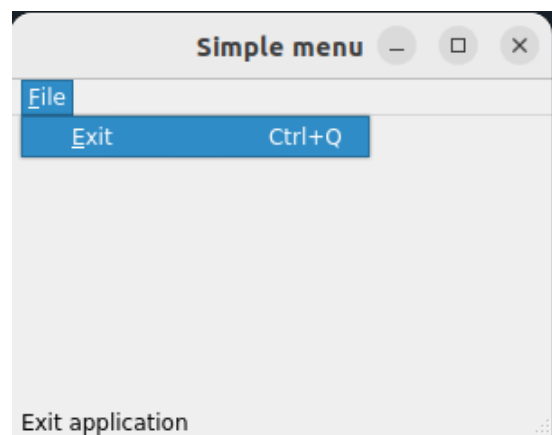


Figura 9: [Ejemplo simple.menu.py](#)

En el ejemplo "submenu.py" nos muestra como crear un menú dentro de una opción de la barra de herramientas, lo cual nos puede ser muy útil para agrupar distintas funcionalidades.

```
import sys
from PyQt5.QtWidgets import QMainWindow, QAction, QMenu, QApplication

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        menubar = self.menuBar()
        fileMenu = menubar.addMenu('File')

        impMenu = QMenu('Import', self)
        impAct = QAction('Import mail', self)
        impMenu.addAction(impAct)

        newAct = QAction('New', self)
        fileMenu.addAction(newAct)
        fileMenu.addMenu(impMenu)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Submenu')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

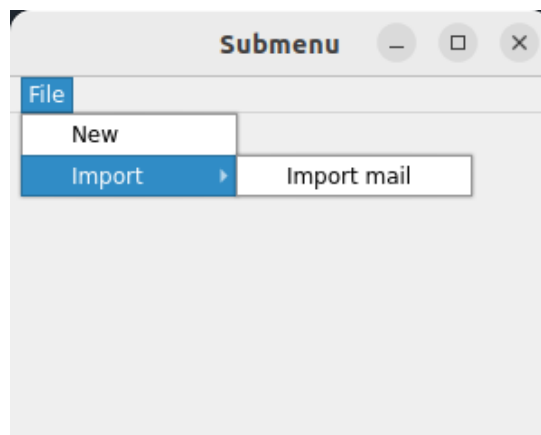


Figura 10: [Ejemplo submenu.py](#)

En el ejemplo "toolbar.py" nos explica como podemos crear una barra de herramientas en las que encontramos imágenes para seleccionar la opción que queremos.

```
import sys
from PyQt5.QtWidgets import QMainWindow, QAction, QApplication
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        exitAct = QAction(QIcon('exit.png'), 'Exit', self)
        exitAct.setShortcut('Ctrl+Q')
        exitAct.triggered.connect(QApp.quit)

        self.toolbar = self.addToolBar("Exit")
        self.toolbar.addAction(exitAct)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Simple menu')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

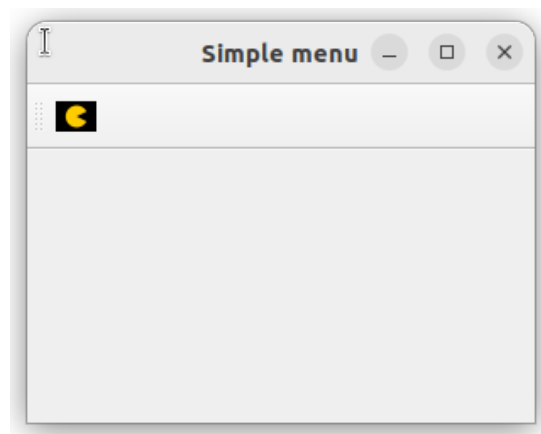


Figura 11: [Ejemplo toolbar.py](#)

En el ejemplo "check.menu.py" nos explica como podemos hacer para que en nuestra barra de herramientas aparezca una opción de marcar y desmarcar con un check, además de marcar como queremos que este en el momento de creación de la ventana.

```

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.statusbar = self.statusBar()
        self.statusbar.showMessage('Ready')

        menubar = self.menuBar()
        viewMenu = menubar.addMenu('View')

        viewStatAct = QAction('View statusbar', self, checkable=True)
        viewStatAct.setStatusTip('View statusbar')
        viewStatAct.setChecked(True)
        viewStatAct.triggered.connect(self.toggleMenu)

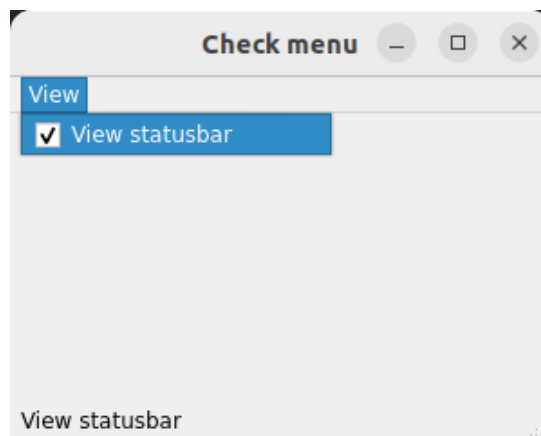
        viewMenu.addAction(viewStatAct)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Check menu')
        self.show()

    def toggleMenu(self, state):
        if state:
            self.statusbar.show()
        else:
            self.statusbar.hide()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Figura 12: [Ejemplo check_menu.py](#)

En el ejemplo "context_menu.py" nos enseña como podemos crear un menú que se muestre en el momento que hagamos click derecho sobre la ventana.

```

import sys
from PyQt5.QtWidgets import QMainWindow, QApplication, QMenu, QAction

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Context menu')
        self.show()

    def contextMenuEvent(self, event):
        cmenu = QMenu(self)

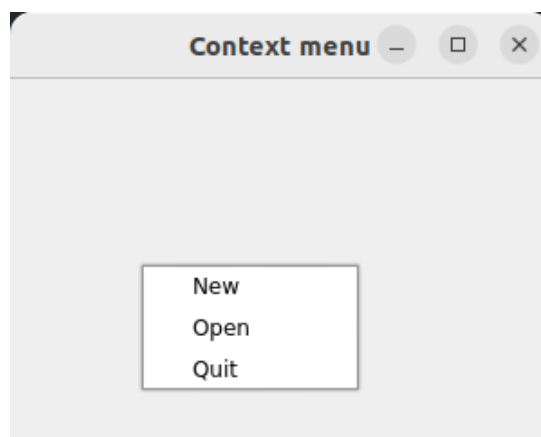
        newAct = cmenu.addAction("New")
        openAct = cmenu.addAction("Open")
        quitAct = cmenu.addAction("Quit")
        action = cmenu.exec_(self.mapToGlobal(event.pos()))

        if action == quitAct:
            QApplication.quit()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()

```

Figura 13: [Ejemplo context_menu.py](#)

En el ejemplo "main_window.py" nos muestra como crear una ventana con todos los elementos vistos en este apartado del tutorial, es decir una barra de herramientas, una barra de herramientas con imágenes y una barra de estados.

```
import sys
from PyQt5.QtWidgets import QMainWindow, QTextEdit, QAction, QApplication
from PyQt5.QtGui import QIcon

class Example(QMainWindow):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        textEdit = QTextEdit()
        self.setCentralWidget(textEdit)

        exitAct = QAction(QIcon('web.png'), 'Exit', self)
        exitAct.setShortcut('Ctrl+Q')
        exitAct.setStatusTip('Exit application')
        exitAct.triggered.connect(self.close)

        self.setStatusBar()

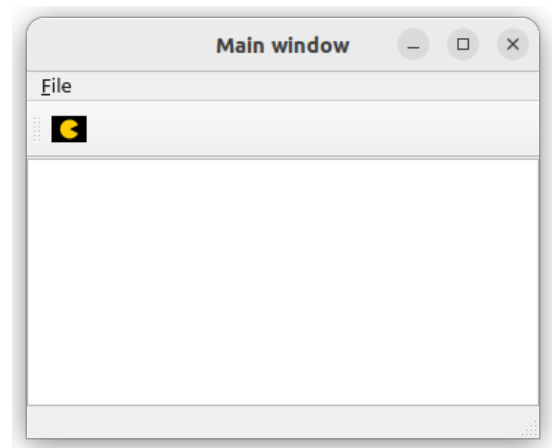
        menubar = self.menuBar()
        fileMenu = menubar.addMenu('&File')
        fileMenu.addAction(exitAct)

        toolbar = self.addToolBar('Exit')
        toolbar.addAction(exitAct)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('Main window')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 14: [Ejemplo main_window.py](#)

1.4. Funciones Importantes

En este apartado vamos a resumir las funciones más importantes de los códigos vistos anteriormente, junto con una breve descripción de lo que hace cada una de ellas:

Función	Descripción
resize()	Cambia el tamaño de la ventana
move()	Cambia la posición en la que se va a situar la ventana
setWindowTitle()	Cambia el título de la ventana
setWindowIcon()	Cambia el icono de la ventana
show()	Muestra la ventana
setGeometry()	Cambia el tamaño y la posición de la ventana
frameGeometry()	Devuelve el tamaño y la posición de la ventana
setToolTip()	Añade un cuadro de texto emergente a la ventana
statusBar()	Crea una barra de estados
menuBar()	Crea una barra de herramientas

Cuadro 1: Funciones de QMainWindow

Función	Descripción
connect()	Conecta un evento al botón
resize()	Cambia el tamaño del botón
sizeHint()	Devuelve el tamaño recomendado de un botón
move()	Cambia la posición del botón
setToolTip()	Añade un cuadro de texto emergente al botón

Cuadro 2: Funciones de QPushButton

Función	Descripción
setShortcut()	Crea un atajo de teclado para realizar una acción
setStatusTip()	Añade un texto a la barra de estados
connect()	Asocia un evento cuando se realiza la acción
setChecked()	Establece si la acción está activa o no

Cuadro 3: Funciones de QAction

Función	Descripción
<code>addMenu()</code>	Añade un menú a la barra de herramientas
<code>addAction()</code>	Añade una acción a la barra de herramientas
<code>exec_()</code>	Crea un menú emergente en la posición del cursor

Cuadro 4: Funciones de QMenu

1.5. Ejemplo propio

En este apartado vamos a ver un ejemplo aplicando la mayoría de funciones aprendidas anteriormente con los diferentes tutoriales, en este caso he hecho una ventana con un botón para salir y un menú con un submenú, que en su interior tiene una opción para salir, además cualquier opción de salir que selecciones sacará una ventana emergente para que confirmes que quieres salir.

```

1  #!/usr/bin/python
2
3  import sys
4  from PyQt5.QtWidgets import QMainWindow, QAction, QMenu, QApplication
5  from PyQt5.QtWidgets import QPushButton, QMessageBox
6
7  class Example(QMainWindow):
8
9      def __init__(self):|
10         super().__init__()
11
12         self.initUI()
13
14     def initUI(self):
15
16         # Configuramos en la barra de herramientas un menu "File",
17         # con un submenu "Opciones" donde tendra una opcion para
18         # cerrar la ventana
19         menubar = self.menuBar()
20         fileMenu = menubar.addMenu('File')
21
22         impMenu = QMenu('Opciones', self)
23         impAct = QAction('Salir', self)
24         impAct.triggered.connect(self.close)
25
26         impMenu.addAction(impAct)
27
28         fileMenu.addMenu(impMenu)
29
30         # Configuramos un boton para cerrar la ventana
31         btn = QPushButton("Salir", self)
32         btn.resize(btn.sizeHint())
33         btn.move(50,50)
34         btn.clicked.connect(self.close)
35         btn.setToolTip("Este boton <b>Sale</b> de la aplicacion")
36
37         self.setGeometry(300, 300, 300, 200)
38         self.setWindowTitle('Ejemplo')
39         self.show()
40
41     #Añadimos una ventana emergente para
42     #asegurarnos que queremos cerrar la ventana
43     def closeEvent(self, event):
44
45         reply = QMessageBox.question(self, 'Message',
46                                     "¿Seguro que quieres salir de la aplicación?",
47                                     QMessageBox.Yes |
48                                     QMessageBox.No, QMessageBox.No)
49
50         if reply == QMessageBox.Yes:
51             event.accept()
52         else:
53             event.ignore()
54
55     def main():
56         app = QApplication(sys.argv)
57         ex = Example()
58         sys.exit(app.exec_())
59
60 if __name__ == '__main__':
61     main()
62
63
64
65

```

Figura 15: [Ejemplo propio](#)

2. Practica 2

2.1. Layout management

En este apartado del tutorial nos explica como podemos posicionar la información que aparece en nuestra ventana:

En el ejemplo "absolute.py" nos muestra como podemos hacer para posicionar cualquier elemento, en este caso texto plano, de forma absoluta, es decir, no importa el tamaño de la ventana este elemento siempre va a estar en la misma posición.

```
import sys
from PyQt5.QtWidgets import QWidget, QLabel, QApplication

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        lbl1 = QLabel('ZetCode', self)
        lbl1.move(15, 10)

        lbl2 = QLabel('tutorials', self)
        lbl2.move(35, 40)

        lbl3 = QLabel('for programmers', self)
        lbl3.move(55, 70)

        self.setGeometry(300, 300, 250, 150)
        self.setWindowTitle('Absolute')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

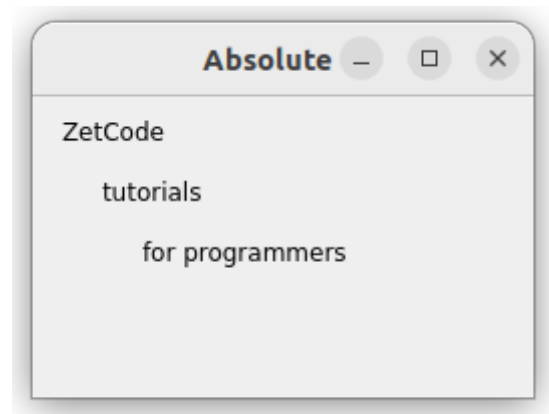


Figura 16: [Ejemplo absolute.py](#)

En el ejemplo "box_layout.py" nos enseña como podemos posicionar cualquier elemento, en este caso dos botones, de forma relativa, es decir, los botones van a moverse en función del tamaño de la ventana.

```

import sys
from PyQt5.QtWidgets import (QWidget, QPushButton,
                              QHBoxLayout, QVBoxLayout, QApplication)

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        okButton = QPushButton("OK")
        cancelButton = QPushButton("Cancel")

        hbox = QHBoxLayout()
        hbox.addStretch(1)
        hbox.addWidget(okButton)
        hbox.addWidget(cancelButton)

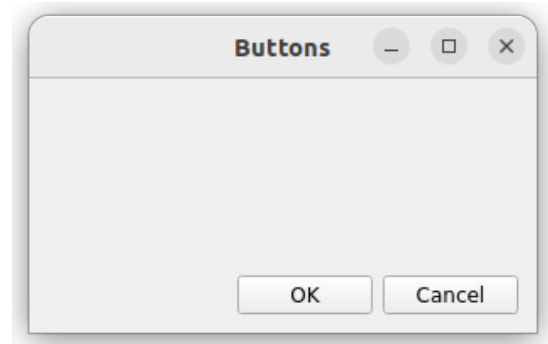
        vbox = QVBoxLayout()
        vbox.addStretch(1)
        vbox.addLayout(hbox)

        self.setLayout(vbox)

        self.setGeometry(300, 300, 300, 150)
        self.setWindowTitle('Buttons')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Figura 17: [Ejemplo box_layout.py](#)

En el ejemplo "calculator.py" nos enseña como podemos usar "QGridLayout" para posicionar los elementos en nuestra ventana, ya que este elemento divide la ventana en filas y columnas y nos permite poner en cada celda uno o varios elementos.

```

import sys
from PyQt5.QtWidgets import (QWidget, QGridLayout,
                              QPushButton, QApplication)

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        grid = QGridLayout()
        self.setLayout(grid)

        names = ['Cls', 'Bck', '', 'Close',
                 '7', '8', '9', '/',
                 '4', '5', '6', '*',
                 '1', '2', '3', '-',
                 '0', '.', '=', '+']

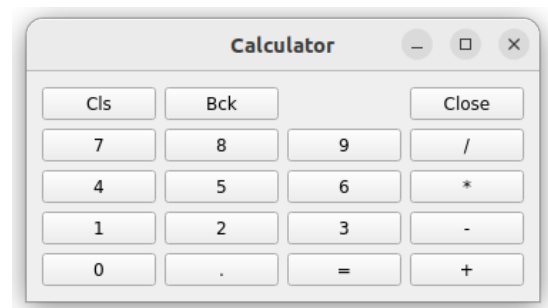
        positions = [(i, j) for i in range(5) for j in range(4)]

        for position, name in zip(positions, names):
            if name == '':
                continue
            button = QPushButton(name)
            grid.addWidget(button, *position)

        self.move(300, 150)
        self.setWindowTitle('Calculator')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Figura 18: [Ejemplo calculator.py](#)

2.2. Events and signals

En este apartado nos enseñan como manejar diferentes eventos y señales para poder realizar ciertas acciones cuando modifiquemos algún elemento de la ventana:

En el ejemplo "signals_slots.py" nos explican el comportamiento básico de los eventos en este caso hacemos un contador que cambia su valor conforme movemos una barra deslizante.

```
import sys
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import (QWidget, QLCDNumber, QSlider,
                             QVBoxLayout, QApplication)

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        lcd = QLCDNumber(self)
        sld = QSlider(Qt.Horizontal, self)

        vbox = QVBoxLayout()
        vbox.addWidget(lcd)
        vbox.addWidget(sld)

        self.setLayout(vbox)
        sld.valueChanged.connect(lcd.display)

        self.setGeometry(300, 300, 250, 150)
        self.setWindowTitle('Signal and slot')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

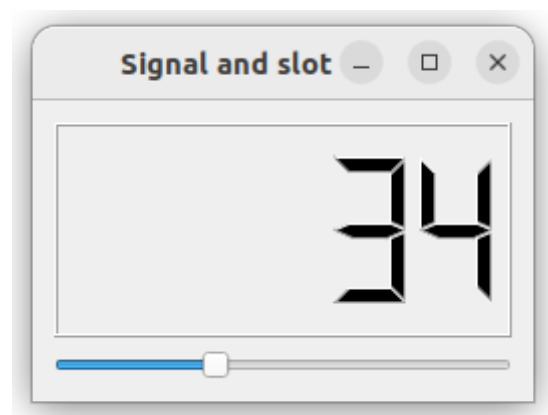


Figura 19: [Ejemplo signals_slots.py](#)

En el ejemplo "reimplement_handler.py" nos enseñan como podemos enlazar un evento al pulsar una tecla, en este caso hacemos que al presionar la tecla `.Escape` se cierre la ventana.

```

import sys
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QWidget, QApplication

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.setGeometry(300, 300, 250, 150)
        self.setWindowTitle('Event handler')
        self.show()

    def keyPressEvent(self, e):
        if e.key() == Qt.Key_Escape:
            self.close()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()

```

Figura 20: [Ejemplo reimplement_handler.py](#)

En el ejemplo "event_object.py" nos explican como podemos enlazar un evento con nuestro ratón, en este caso hacemos que en nuestra ventana se muestren las coordenadas en las que se encuentra el ratón actualmente.

```

import sys
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QWidget, QApplication, QGridLayout, QLabel

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        grid = QGridLayout()

        x = 0
        y = 0

        self.text = f'x: {x}, y: {y}'

        self.label = QLabel(self.text, self)
        grid.addWidget(self.label, 0, 0, Qt.AlignTop)

        self.setMouseTracking(True)

        self.setLayout(grid)

        self.setGeometry(300, 300, 450, 300)
        self.setWindowTitle('Event object')
        self.show()

    def mouseMoveEvent(self, e):
        x = e.x()
        y = e.y()

        text = f'x: {x}, y: {y}'
        self.label.setText(text)

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

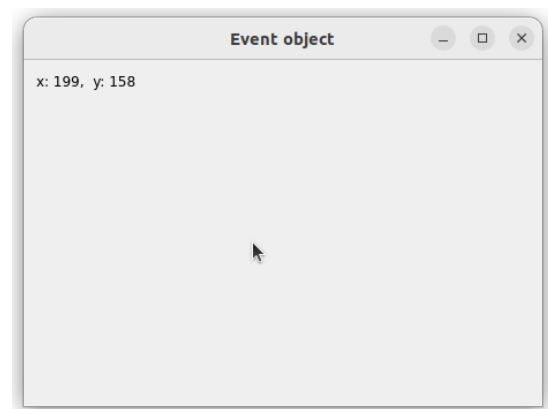


Figura 21: [Ejemplo event_object.py](#)

En el ejemplo "event_sender.py" nos enseñan como podemos enlazar un evento cuando se pulsa un botón, en este caso hacemos que se muestre el botón que se ha pulsado en la barra de estados.

```
import sys
from PyQt5.QtWidgets import QMainWindow, QPushButton, QApplication

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        btn1 = QPushButton("Button 1", self)
        btn1.move(30, 50)

        btn2 = QPushButton("Button 2", self)
        btn2.move(150, 50)

        btn1.clicked.connect(self.buttonClicked)
        btn2.clicked.connect(self.buttonClicked)

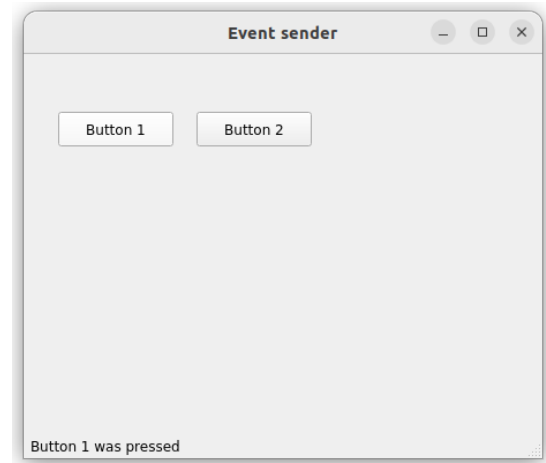
        self.statusBar()

        self.setGeometry(300, 300, 450, 350)
        self.setWindowTitle('Event sender')
        self.show()

    def buttonClicked(self):
        sender = self.sender()
        self.statusBar().showMessage(sender.text() + ' was pressed')

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 22: [Ejemplo event_sender.py](#)

2.3. Funciones Importantes

En este apartado vamos a resumir las funciones más importantes de los códigos vistos anteriormente, junto con una breve descripción de lo que hace cada una de ellas:

Función	Descripción
addStretch()	Añade un espacio flexible dentro del diseño
addWidget()	Añade un widget al diseño
addLayout()	Añade otro diseño dentro del diseño principal

Cuadro 5: Funciones de QHBoxLayout/QVBoxLayout

Función	Descripción
setLayout()	Asigna un diseño a la ventana
close()	Cierra la ventana
setMouseTracking()	Habilita o desactiva el seguimiento del ratón dentro de la ventana

Cuadro 6: Funciones de QWidget

Función	Descripción
sender()	Devuelve información del botón que ha enviado la señal

Cuadro 7: Funciones de QPushButton

2.4. Editor de texto

En este apartado he realizado un editor de texto básico, con las opciones de seleccionar una carpeta, abrir un archivo, guardar un archivo, guardar como un archivo, cerrar una carpeta y salir del editor de texto.

Para ello he estructurado la ventana mediante "QGridLayout" ya que me da la posibilidad de situar los botones y cuadros de texto en la ventana con mayor facilidad que "QHBoxLayout" y "QVBoxLayout" ya que al usar una disposición de filas y columnas, puedo colocar cada elemento con mayor precisión y facilidad. Junto a esto el uso de "QGridLayout" también da la ventaja de poder ajustar con mayor facilidad el tamaño de cada elemento.

```
def initUI(self):
    self.opened_file = ""

    # Creación de elementos de la ventana
    txtFolder = QLabel('Carpeta:', self)

    folderButton = QPushButton("Carpeta")
    openButton = QPushButton("Abrir")
    saveButton = QPushButton("Guardar")
    saveAsButton = QPushButton("Guardar como")
    closeButton = QPushButton("Cerrar")
    exitButton = QPushButton("Salir")

    self.lineFolder = QLineEdit("")
    self.lineFolder.setReadOnly(True)
    self.listFiles = QListWidget()
    self.lineEdit = QTextEdit("")

    # Posicionamos los elementos creados anteriormente
    gridla = QGridLayout(self)
    gridla.addWidget(txtFolder, 1, 1, 1, 1)
    gridla.addWidget(self.lineFolder, 1, 2, 1, 24)
    gridla.addWidget(self.listFiles, 2, 1, 24, 5)
    gridla.addWidget(self.lineEdit, 2, 6, 24, 20)

    gridla.addWidget(folderButton, 1, 26, 1, 4)
    gridla.addWidget(openButton, 2, 26, 1, 4)
    gridla.addWidget(saveButton, 3, 26, 1, 4)
    gridla.addWidget(saveAsButton, 4, 26, 1, 4)
    gridla.addWidget(closeButton, 5, 26, 1, 4)
    gridla.addWidget(exitButton, 6, 26, 1, 4)

    # Añadimos acciones a los botones
    exitButton.clicked.connect(QApplication.instance().quit)
    folderButton.clicked.connect(self.openFolder)
    closeButton.clicked.connect(self.closeFolder)
    openButton.clicked.connect(self.openFile)
    self.listFiles.itemDoubleClicked.connect(self.openFile)
    saveButton.clicked.connect(self.saveFile)
    saveAsButton.clicked.connect(self.saveAsFile)

    self.setGeometry(300, 300, 600, 600)
    self.setWindowTitle('Editor de texto')
    self.show()
```

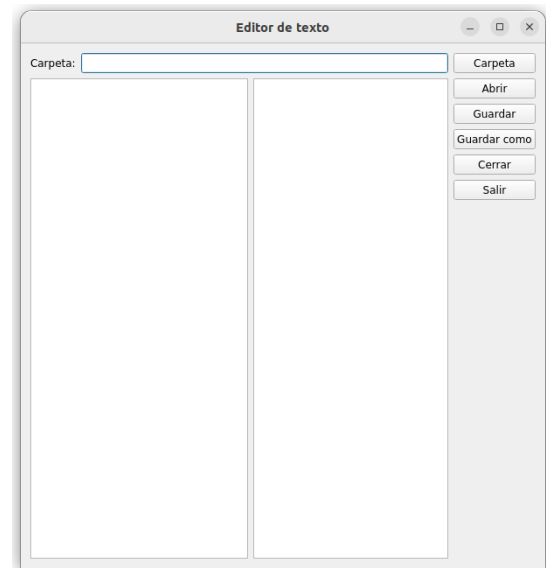


Figura 23: [Ejemplo editor_texto.py](#)

Además se le han asociado diferentes eventos a cada botón para que la aplicación sea funcional y pueda hacer todas las operaciones básicas que debes poder hacer en un editor de textos.

```
def openFolder(self):
    folder = QFileDialog.getExistingDirectory(self, "Seleccionar Carpeta")

    if folder:
        self.lineFolder.clear()
        self.lineFolder.setText(folder)
        self.listFiles.clear()
        # Filtrar solo archivos
        files = [f for f in os.listdir(folder) if os.path.isfile(os.path.join(folder, f))]
        self.listFiles.addItem(files)

def closeFolder(self):
    self.lineFolder.clear()
    self.listFiles.clear()
    self.lineText.clear()
    self.opened_file = ""

def openFile(self):
    self.opened_file = self.listFiles.currentItem()
    if self.opened_file:
        with open(os.path.join(self.lineFolder.text(), self.opened_file.text()), "r", encoding="utf-8") as file:
            content = file.read()
            self.lineText.setText(content)

def saveFile(self):
    if self.opened_file:
        with open(os.path.join(self.lineFolder.text(), self.opened_file.text()), "w", encoding="utf-8") as file:
            file.write(self.lineText.toPlainText())

def saveAsFile(self):
    if self.lineText.toPlainText():
        options = QFileDialog.Options()
        # Ponemos como segunda variable _ para ignorar lo que devuelve
        file_name, _ = QFileDialog.getSaveFileName(self, "Guardar Como", "", "Archivos de texto (*.txt);;Todos los archivos (*)",
        if file_name:
            with open(file_name, "w", encoding="utf-8") as file:
                file.write(self.lineText.toPlainText())
```

Figura 24: [Eventos ejemplo editor_texto.py](#)

2.5. Aplicación básica

En este apartado he realizado una aplicación básica en la que podemos encontrar una barra de estados que siempre muestra el mensaje 'Esta es la barra de estados' a no ser que se desactive la opción que se encuentra en el menú 'File' que nos permite visualizar la barra de estados. Además, tiene una opción en la barra de herramientas que si la pulsamos cierra la ventana, al igual que el botón de salir y, por último, tiene un botón 'Hola' que cuando lo pulsamos nos crea una ventana emergente con un saludo.

```

7 class Example(QMainWindow):
8
9     def __init__(self):
10         super().__init__()
11
12         self.initUI()
13
14     def initUI(self):
15
16         # Creamos la barra de estados y hacemos que siempre tenga un mensaje
17         self.statusbar = self.statusBar()
18         self.statusbar.showMessage("Esta es la barra de estados")
19
20         # Creamos una barra de herramientas y le ponemos una opción para salir
21         exitActToolBar = QAction(QIcon('exit.png'), '&Exit', self)
22         exitActToolBar.triggered.connect(QApplication.instance().quit)
23         toolbar = self.addToolBar('Exit')
24         toolbar.addAction(exitActToolBar)
25
26         # Creamos un menu de opciones que contiene un submenu para ocultar la barra de estados
27         # y tambien tiene una opcion para salir
28         menubar = self.menuBar()
29         fileMenu = menubar.addMenu('File')
30
31         statAct = QAction('Ver barra de estados', self, checkable=True)
32         statAct.setChecked(True)
33         statAct.triggered.connect(self.toggleMenu)
34         exitActMenu = QAction('Salir', self)
35         exitActMenu.triggered.connect(self.close)
36
37         impMenu = QMenu('Opciones', self)
38         impMenu.addAction(statAct)
39
40         fileMenu.addMenu(impMenu)
41         fileMenu.addAction(exitActMenu)
42
43         # Configuramos un boton para cerrar la ventana
44         btn = QPushButton("Salir", self)
45         btn.resize(btn.sizeHint())
46         btn.clicked.connect(self.close)
47         btn.setToolTip("Este boton <b>Sale</b> de la aplicacion")
48
49         # Configuramos un boton que nos cree una ventana emergente con un mensaje
50         btn2 = QPushButton("Hola", self)
51         btn2.resize(btn2.sizeHint())
52         btn2.clicked.connect(self.saludo)
53
54         # Utilizamos el widget para poder posicionar los botones ya que en un QMainWindow
55         # no se puede utilizar QVBoxLayout/QHBoxLayout
56         central_widget = QWidget()
57         self.setCentralWidget(central_widget)
58
59         # Posicionamos el boton de Hola arriba a la derecha y el de Salir abajo a la derecha
60         vbox = QVBoxLayout()
61         vbox.addWidget(btn2)
62         vbox.addStretch(1)
63         vbox.addWidget(btn)
64
65         hbox = QHBoxLayout()
66         hbox.addStretch(1)
67         hbox.addLayout(vbox)
68
69         central_widget.setLayout(hbox)
70
71         self.setGeometry(300, 300, 300, 200)
72         self.setWindowTitle('Aplicacion Basica')
73         self.show()
74
75     def toggleMenu(self, state):
76         if state:
77             self.statusbar.show()
78             self.statusbar.showMessage("Esta es la barra de estados")
79         else:
80             self.statusbar.hide()
81
82     def saludo(self):
83         QMessageBox.about(self, 'Saludo', 'Hola, ¿como estas?')
84

```

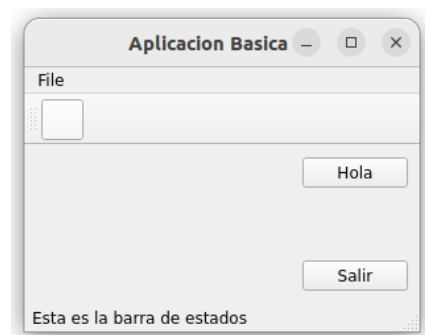


Figura 25: [Ejemplo aplicacion_basica.py](#)

3. Practica 3

3.1. Dialogs

En este apartado nos muestran como podemos hacer para añadir diferentes ventanas de diálogo para poder comunicarnos con el programa de diferentes maneras:

En el ejemplo "input_dialog.py" nos enseñan como podemos crear un cuadro de diálogo en el que podemos introducir texto y en este caso que el texto que introduzcamos se muestre en nuestra ventana.

```
from PyQt5.QtWidgets import (QWidget, QPushButton, QLineEdit,
                             QInputDialog, QApplication)
import sys

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.btn = QPushButton('Dialog', self)
        self.btn.move(20, 20)
        self.btn.clicked.connect(self.showDialog)

        self.le = QLineEdit(self)
        self.le.move(130, 22)

        self.setGeometry(300, 300, 450, 350)
        self.setWindowTitle('Input dialog')
        self.show()

    def showDialog(self):
        text, ok = QInputDialog.getText(self, 'Input Dialog',
                                      'Enter your name:')

        if ok:
            self.le.setText(str(text))

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

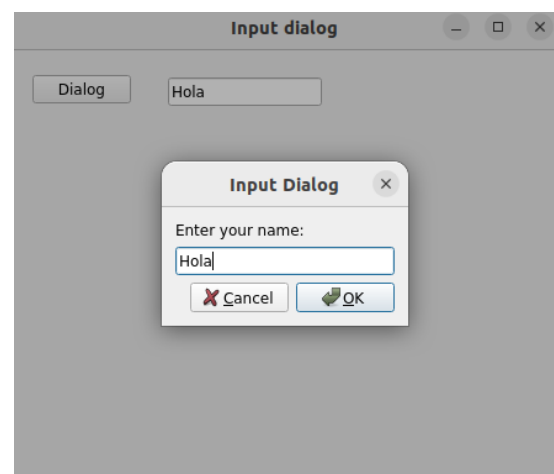


Figura 26: [Ejemplo input_dialog.py](#)

En el ejemplo "color_dialog.py" nos explican como añadir un cuadro de diálogo en el que podemos seleccionar un color.


```

from PyQt5.QtWidgets import (QWidget, QPushButton, QFrame,
                             QColorDialog, QApplication)
from PyQt5.QtGui import QColor
import sys

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        col = QColor(0, 0, 0)

        self.btn = QPushButton('Dialog', self)
        self.btn.move(20, 20)

        self.btn.clicked.connect(self.showDialog)

        self.frm = QFrame(self)
        self.frm.setStyleSheet('QWidget { background-color: %s }'
                               % col.name())
        self.frm.setGeometry(130, 22, 200, 200)

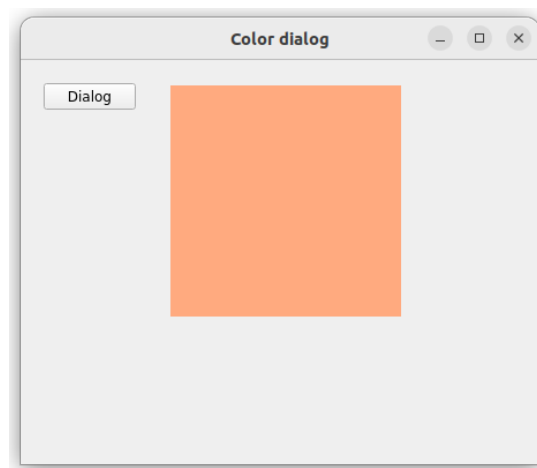
        self.setGeometry(300, 300, 450, 350)
        self.setWindowTitle('Color dialog')
        self.show()

    def showDialog(self):
        col = QColorDialog.getColor()

        if col.isValid():
            self.frm.setStyleSheet('QWidget { background-color: %s }'
                                   % col.name())

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Figura 27: [Ejemplo color_dialog.py](#)

En el ejemplo "font_dialog.py" nos enseñan como hacer un cuadro de diálogo que nos permita cambiar el formato del texto, en este cuadro podemos modificar tanto el tipo de letra, como el tamaño de la misma, además de permitirnos añadir un subrayado.

```

from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QPushButton,
                             QSizePolicy, QLabel, QFontDialog, QApplication)
import sys

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        vbox = QVBoxLayout()

        btn = QPushButton('Dialog', self)
        btn.setSizePolicy(QSizePolicy.Fixed, QSizePolicy.Fixed)
        btn.move(20, 20)

        vbox.addWidget(btn)

        btn.clicked.connect(self.showDialog)

        self.lbl = QLabel('Knowledge only matters', self)
        self.lbl.move(130, 20)

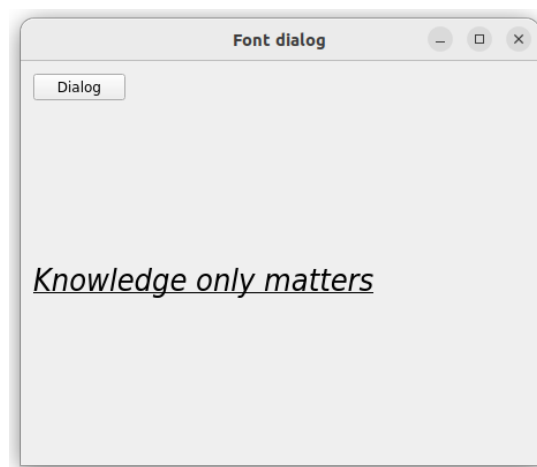
        vbox.addWidget(self.lbl)
        self.setLayout(vbox)

        self.setGeometry(300, 300, 450, 350)
        self.setWindowTitle('Font dialog')
        self.show()

    def showDialog(self):
        font, ok = QFontDialog.getFont()
        if ok:
            self.lbl.setFont(font)

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Figura 28: [Ejemplo font_dialog.py](#)

En el ejemplo "file_dialog.py" nos explican como podemos crear un cuadro de diálogo que nos permita

seleccionar un fichero de nuestro ordenador para abrirlo, en este caso para mostrar la información que contiene.

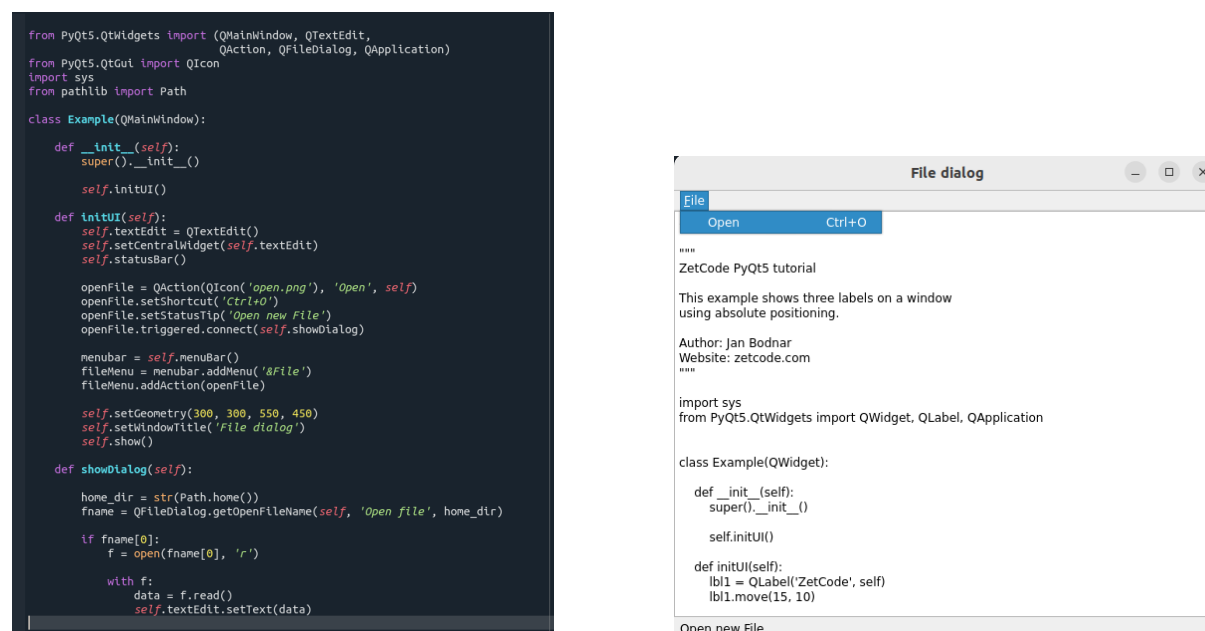


Figura 29: [Ejemplo file_dialog.py](#)

3.2. Widgets

En este apartado vamos a ver diferentes Widgets en nuestra ventana, a la vez que le asignamos diferentes funcionalidades que reaccionen con la ventana.

En el ejemplo "check_box.py" creamos una check box, la cual cuando está marcada nos muestra el título de la ventana y cuando está deshabilitada nos oculta el título de la ventana.

```
from PyQt5.QtWidgets import QWidget, QCheckBox, QApplication
from PyQt5.QtCore import Qt
import sys

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        cb = QCheckBox('Show title', self)
        cb.move(20, 20)
        cb.toggle()
        cb.stateChanged.connect(self.changeTitle)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('QCheckBox')
        self.show()

    def changeTitle(self, state):

        if state == Qt.Checked:
            self.setWindowTitle('QCheckBox')
        else:
            self.setWindowTitle(' ')

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

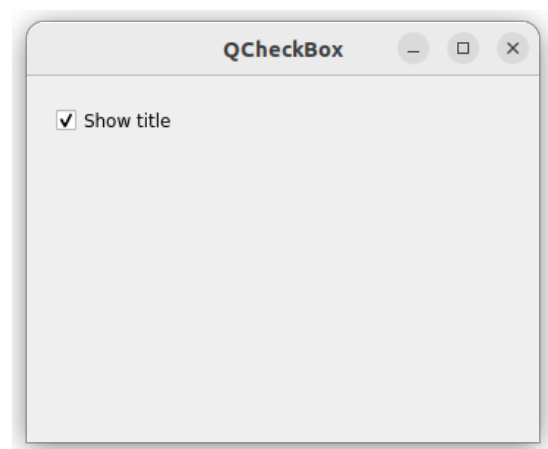


Figura 30: [Ejemplo check_box.py](#)

En el ejemplo "toggle_button.py" nos explica como crear un cuadro de color el cual va mostrando diferentes colores en función de los botones que hemos pulsado, pudiendo mostrar mezclas de los colores propuestos en los botones.

```

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.col = QColor(0, 0, 0)

        redb = QPushButton('Red', self)
        redb.setCheckable(True)
        redb.move(10, 10)

        redb.clicked[bool].connect(self.setColor)

        greenb = QPushButton('Green', self)
        greenb.setCheckable(True)
        greenb.move(10, 60)

        greenb.clicked[bool].connect(self.setColor)

        blueb = QPushButton('Blue', self)
        blueb.setCheckable(True)
        blueb.move(10, 110)

        blueb.clicked[bool].connect(self.setColor)

        self.square = QFrame(self)
        self.square.setGeometry(150, 20, 100, 100)
        self.square.setStyleSheet("QWidget { background-color: %s }" %
                                   self.col.name())

        self.setGeometry(300, 300, 300, 250)
        self.setWindowTitle('Toggle button')
        self.show()

    def setColor(self, pressed):
        source = self.sender()

        if pressed:
            val = 255
        else:
            val = 0

        if source.text() == "Red":
            self.col.setRed(val)
        elif source.text() == "Green":
            self.col.setGreen(val)
        else:
            self.col.setBlue(val)

        self.square.setStyleSheet("QFrame { background-color: %s }" %
                                   self.col.name())

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()

```

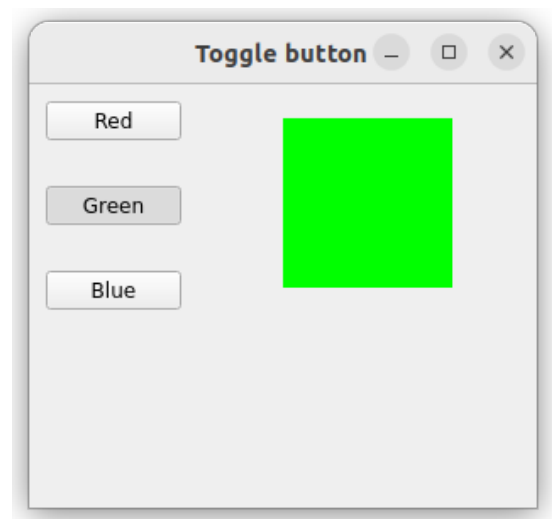


Figura 31: [Ejemplo toggle.button.py](#)

En el ejemplo "slider.py" nos enseñan como podemos crear una barra deslizable la cual va cambiando la imagen que se muestra al lado de ella en función del valor que esté marcando la barra deslizable.

```

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

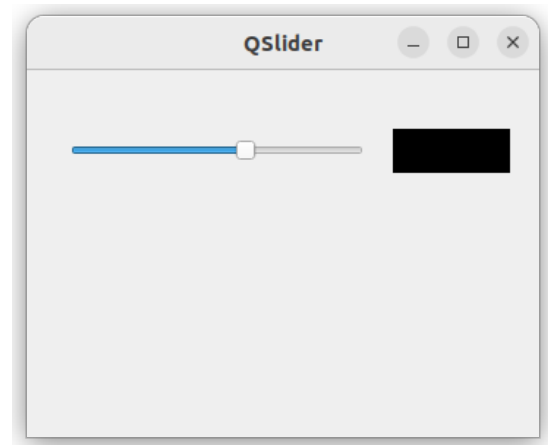
    def initUI(self):
        sld = QSlider(Qt.Horizontal, self)
        sld.setFocusPolicy(Qt.NoFocus)
        sld.setGeometry(30, 40, 200, 30)
        sld.valueChanged[int].connect(self.changeValue)

        self.label = QLabel(self)
        self.label.setPixmap(QPixmap('mute.png'))
        self.label.setGeometry(250, 40, 80, 30)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('QSlider')
        self.show()

    def changeValue(self, value):
        if value == 0:
            self.label.setPixmap(QPixmap('mute.png'))
        elif 0 < value <= 30:
            self.label.setPixmap(QPixmap('min.png'))
        elif 30 < value < 80:
            self.label.setPixmap(QPixmap('med.png'))
        else:
            self.label.setPixmap(QPixmap('max.png'))

```

Figura 32: [Ejemplo slider.py](#)

En el ejemplo "progressbar.py" nos explican como podemos crear una barra de progreso que empieza a avanzar cuando pulsamos el botón y si lo volvemos a pulsar mientras esta avanzando se detiene.

```

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.pbar = QProgressBar(self)
        self.pbar.setGeometry(30, 40, 200, 25)

        self.btn = QPushButton('Start', self)
        self.btn.move(40, 80)
        self.btn.clicked.connect(self.doAction)

        self.timer = QTimer()
        self.step = 0

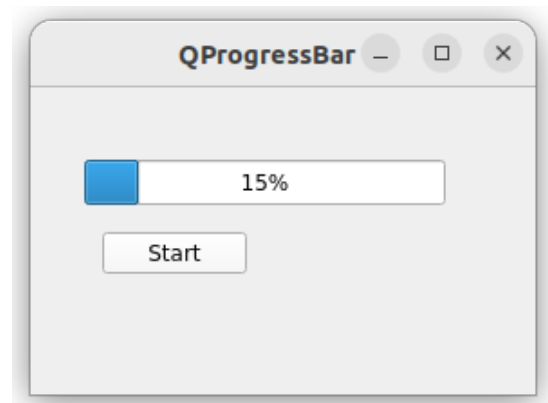
        self.setGeometry(300, 300, 280, 170)
        self.setWindowTitle('QProgressBar')
        self.show()

    def timerEvent(self, e):
        if self.step >= 100:
            self.timer.stop()
            self.btn.setText('Finished')
            return

        self.step = self.step + 1
        self.pbar.setValue(self.step)

    def doAction(self):
        if self.timer.isActive():
            self.timer.stop()
            self.btn.setText('Start')
        else:
            self.timer.start(100, self)
            self.btn.setText('Stop')

```

Figura 33: [Ejemplo progressbar.py](#)

En el ejemplo "calendar.py" nos enseñan como crear un calendario en nuestra ventana, en este caso además hacemos que se muestre el día y la fecha debajo del calendario.

```
class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        vbox = QVBoxLayout(self)

        cal = QCalendarWidget(self)
        cal.setGridVisible(True)
        cal.clicked[QDate].connect(self.showDate)

        vbox.addWidget(cal)

        self.lbl = QLabel(self)
        date = cal.selectedDate()
        self.lbl.setText(date.toString())

        vbox.addWidget(self.lbl)

        self.setLayout(vbox)

        self.setGeometry(300, 300, 350, 300)
        self.setWindowTitle('Calendar')
        self.show()

    def showDate(self, date):
        self.lbl.setText(date.toString())

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

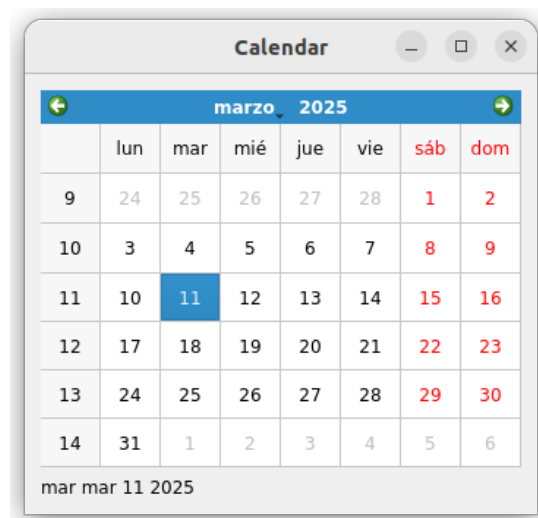


Figura 34: [Ejemplo calendar.py](#)

En el ejemplo "pixmap.py" nos enseñan como podemos añadir una imagen a nuestra ventana a través de "QPixmap".

```
from PyQt5.QtWidgets import (QWidget, QHBoxLayout,
                             QLabel, QApplication)
from PyQt5.QtGui import QPixmap
import sys

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        hbox = QHBoxLayout(self)
        pixmap = QPixmap('sid.jpg')

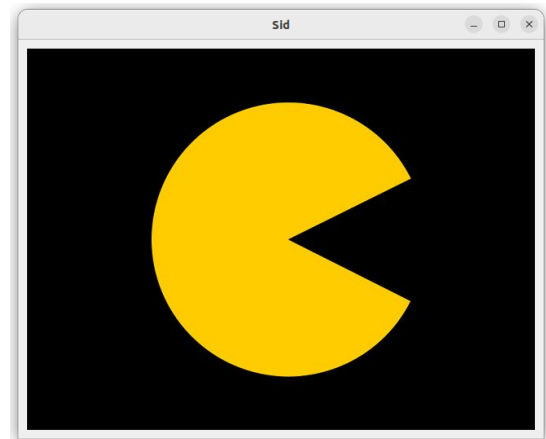
        lbl = QLabel(self)
        lbl.setPixmap(pixmap)

        hbox.addWidget(lbl)
        self.setLayout(hbox)

        self.move(300, 200)
        self.setWindowTitle('sid')
        self.show()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 35: [Ejemplo pixmap.py](#)

En el ejemplo "line.edit.py" nos enseña como podemos añadir una linea de texto en la que podemos escribir cualquier tipo de información que queramos.

```
import sys
from PyQt5.QtWidgets import (QWidget, QLabel,
                             QLineEdit, QApplication)

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.lbl = QLabel(self)
        qle = QLineEdit(self)

        qle.move(60, 100)
        self.lbl.move(60, 40)

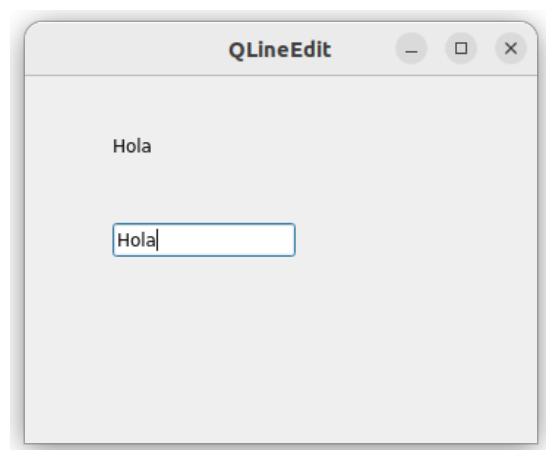
        qle.textChanged[str].connect(self.onChanged)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('QLineEdit')
        self.show()

    def onChanged(self, text):
        self.lbl.setText(text)
        self.lbl.adjustSize()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

Figura 36: [Ejemplo line.edit.py](#)

En el ejemplo "splitter.py" nos explica como podemos hacer para crear distintos límites dentro de nuestra

ventana, estos límites pueden variar de tamaño arrastrándolos con el ratón, así ajustándolos a la ventana de la manera que prefieras.

```
class Example(QWidget):  
    def __init__(self):  
        super().__init__()  
        self.initUI()  
    def initUI(self):  
        hbox = QHBoxLayout(self)  
  
        topleft = QFrame(self)  
        topleft.setFrameShape(QFrame.StyledPanel)  
  
        topright = QFrame(self)  
        topright.setFrameShape(QFrame.StyledPanel)  
  
        bottom = QFrame(self)  
        bottom.setFrameShape(QFrame.StyledPanel)  
  
        splitter1 = QSplitter(Qt.Horizontal)  
        splitter1.addWidget(topleft)  
        splitter1.addWidget(topright)  
  
        splitter2 = QSplitter(Qt.Vertical)  
        splitter2.addWidget(splitter1)  
        splitter2.addWidget(bottom)  
  
        hbox.addWidget(splitter2)  
        self.setLayout(hbox)  
  
        self.setGeometry(300, 300, 450, 400)  
        self.setWindowTitle('QSplitter')  
        self.show()  
  
def main():  
    app = QApplication(sys.argv)  
    ex = Example()  
    sys.exit(app.exec_())
```



Figura 37: [Ejemplo splitter.py](#)

En el ejemplo "combobox.py" nos explican como podemos añadir un listado con varias opciones a nuestra ventana, además en este caso hacemos que la opción seleccionada se muestre en un texto debajo del selector.


```
import sys
from PyQt5.QtWidgets import (QWidget, QLabel,
                              QComboBox, QApplication)

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        self.lbl = QLabel('Ubuntu', self)

        combo = QComboBox(self)
        combo.addItem('Ubuntu')
        combo.addItem('Mandriva')
        combo.addItem('Fedora')
        combo.addItem('Arch')
        combo.addItem('Gentoo')

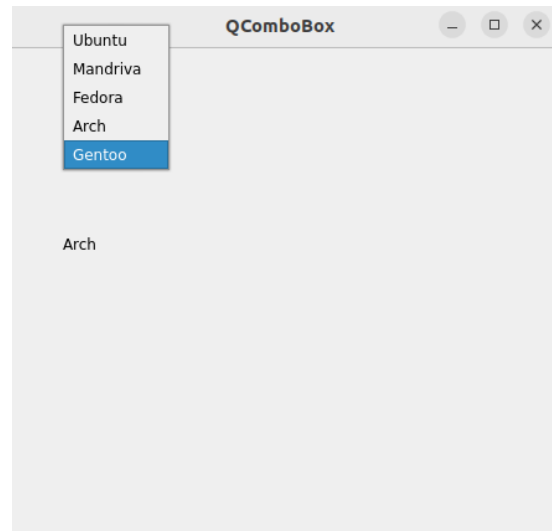
        combo.move(50, 50)
        self.lbl.move(50, 150)

        combo.activated[str].connect(self.onActivated)

        self.setGeometry(300, 300, 450, 400)
        self.setWindowTitle('QComboBox')
        self.show()

    def onActivated(self, text):
        self.lbl.setText(text)
        self.lbl.adjustSize()

def main():
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Figura 38: [Ejemplo combobox.py](#)

3.3. Funciones Importantes

En este apartado vamos a resumir las funciones más importantes de los códigos vistos anteriormente, junto con una breve descripción de lo que hace cada una de ellas:

Función	Descripción
QLineEdit()	Crea un campo de texto
QColor()	Crea un menu de selección de color
QFontDialog()	Crea un menu para seleccionar la fuente y el tamaño de un texto
QFileDialog()	Crea un menu para seleccionar un fichero

Cuadro 8: Creación de Dialogs

Función	Descripción
QCheckBox()	Crea un cuadro de selección que permite elegir entre dos opciones (marcado o desmarcado)
QSlider()	Crea un control deslizante que permite seleccionar un valor dentro de un rango
QProgressBar()	Crea una barra de progreso
QCalendarWidget()	Crea un widget de calendario para seleccionar fechas
QPixmap()	Representa una imagen que se puede mostrar en la interfaz gráfica
QFrame()	Crea un marco que puede usarse para agrupar y estilizar otros widgets
QComboBox()	Proporciona una lista desplegable para seleccionar una opción de un conjunto de elementos

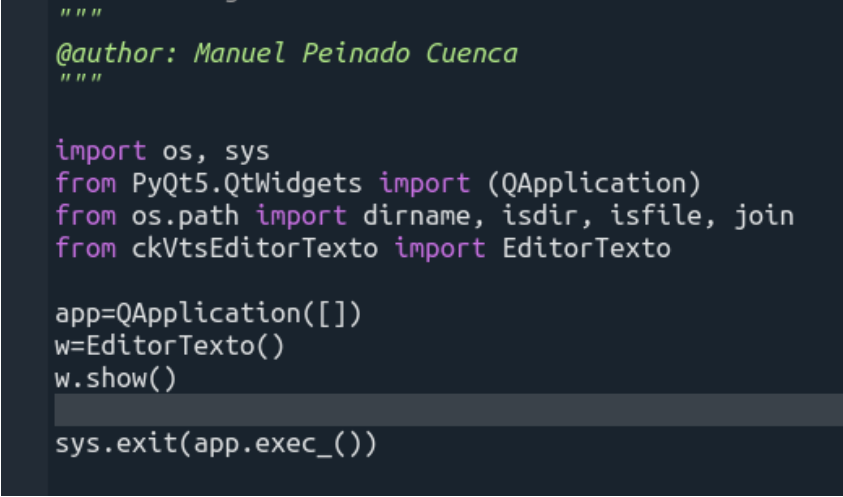
Cuadro 9: Creación de Widgets

3.4. Editor de texto MVC

En este apartado veremos como hacer un editor de texto mediante el modelo vista controlador, para ello he modificado el Editor de texto de la Subsección 2.4, y lo he dividido en los siguientes ficheros:

3.4.1. ckAppEditorTexto

Este fichero actúa como punto de entrada en un modelo Vista-Controlador (MVC). Se encarga de inicializar la aplicación, crear una instancia de QApplication y lanzar la vista (EditorTexto).



```
"""
@author: Manuel Peinado Cuenca
"""

import os, sys
from PyQt5.QtWidgets import QApplication
from os.path import dirname, isdir, isfile, join
from ckVtsEditorTexto import EditorTexto

app=QApplication([])
w=EditorTexto()
w.show()

sys.exit(app.exec_())
```

Figura 39: [ckAppEditorTexto.py](#)

3.4.2. ckVtsEditorTexto

Este fichero actúa como la Vista. Define la interfaz gráfica y gestiona las interacciones del usuario, como abrir carpetas, cargar y guardar archivos. Los eventos de la vista son manejados por el Controlador, que se encarga de la lógica de negocio y la manipulación de los archivos.

```

import os, sys
from PyQt5.QtWidgets import (QWidget, QPushButton, QLabel, QLineEdit, QHBoxLayout,
                             QVBoxLayout, QGridLayout, QApplication, QTextEdit, QListWidget, QFileDialog)

from os.path import dirname, isdir, isfile, join
import ckCtrlEditorText as ctrl

class EditorText(QWidget):
    def __init__(self):
        super(EditorText, self).__init__()

        self.opened_file = ""

        # Creación de elementos de la ventana
        txtFolder = QLabel('Carpeta:', self)

        folderButton = QPushButton("Carpeta")
        openButton = QPushButton("Abrir")
        saveButton = QPushButton("Guardar")
        saveAsButton = QPushButton("Guardar como")
        closeButton = QPushButton("Cerrar")
        exitButton = QPushButton("Salir")

        self.lneFolder = QLineEdit("")
        self.lneFolder.setReadOnly(True)
        self.listFile = QListWidget()
        self.lneText = QTextEdit("")

        # Posicionamos los elementos creados anteriormente
        gridla = QGridLayout(self)
        gridla.addWidget(txtFolder, 1, 1, 1, 1)
        gridla.addWidget(self.lneFolder, 1, 2, 1, 24)
        gridla.addWidget(self.listFile, 2, 1, 24, 5)
        gridla.addWidget(self.lneText, 2, 6, 24, 20)

        gridla.addWidget(folderButton, 1, 26, 1, 4)
        gridla.addWidget(openButton, 2, 26, 1, 4)
        gridla.addWidget(saveButton, 3, 26, 1, 4)
        gridla.addWidget(saveAsButton, 4, 26, 1, 4)
        gridla.addWidget(closeButton, 5, 26, 1, 4)
        gridla.addWidget(exitButton, 6, 26, 1, 4)

        # Añadimos acciones a los botones
        exitButton.clicked.connect(QApplication.instance().quit)
        folderButton.clicked.connect(self.openFolder)

        closeButton.clicked.connect(self.closeFolder)
        openButton.clicked.connect(self.openFile)
        self.listFile.itemDoubleClicked.connect(self.openFile)
        saveButton.clicked.connect(self.saveFile)
        saveAsButton.clicked.connect(self.saveAsFile)

        self.setGeometry(300, 300, 600, 600)
        self.setWindowTitle('Editor de texto')
        self.show()

    def openFolder(self):
        folder = QFileDialog.getExistingDirectory(self, "Seleccionar Carpeta")

        if folder:
            self.lneFolder.clear()
            self.lneFolder.setText(folder)
            self.listFile.clear()
            # Filtrar solo archivos
            files = [f for f in os.listdir(folder) if os.path.isfile(os.path.join(folder, f))]
            self.listFile.addItem(files)

    def closeFolder(self):
        self.lneFolder.clear()
        self.listFile.clear()
        self.lneText.clear()
        self.opened_file = ""

    def openFile(self):
        self.opened_file = self.listFile.currentItem()
        if self.opened_file:
            self_path = os.path.join(self.lneFolder.text(), self.opened_file.text())
            text = ctrl.readEvent(self_path)
            self.lneText.setText(text)

    def saveFile(self):
        if self.opened_file:
            self_path = os.path.join(self.lneFolder.text(), self.opened_file.text())
            text = self.lneText.toPlainText()
            ctrl.saveFileEvent(self_path, text)

    def saveAsFile(self):
        if self.lneText.toPlainText():
            options = QFileDialog.Options()
            # Ponemos como segunda variable _ para ignorar lo que devuelve
            file_name, _ = QFileDialog.getSaveFileName(self, "Guardar Como", "", "Archivos de texto (*.txt);Todos los archivos (*)", options)
            text = self.lneText.toPlainText()
            ctrl.saveFileEvent(file_name, text)

if __name__ == '__main__':
    app = QApplication([])
    w = EditorText()
    w.show()

    sys.exit(app.exec_())

```

Figura 40: [ckVtsEditorText.py](#)

3.4.3. ckModAppEditorTexto

Este fichero actúa como Modelo. Gestiona la lectura y escritura de archivos mediante las funciones "saveFile" y "readText", proporcionando la funcionalidad de manipulación de datos, que luego puede ser utilizada por el Controlador para interactuar con la Vista

```
"""
@author: Manuel Peinado Cuenca
"""

def saveFile(fileName, text):
    f = open(fileName, 'w')
    f.write(text)
    f.close()
    return True

def readText(fileName):
    f = open(fileName, 'r')
    text=f.read()
    f.close()
    return text

if __name__=="__main__":
    pass
```

Figura 41: [ckModAppEditorTexto.py](#)

3.4.4. ckCtrlEditorTexto

Este fichero actúa como Controlador. Gestiona los eventos de la aplicación relacionados con los archivos: llama a las funciones del Modelo (en el módulo ckModAppEditorTexto) para guardar y leer el contenido de los archivos, actuando como intermediario entre la Vista y el Modelo.

```
"""
@author: Manuel Peinado Cuenca
"""

import ckModAppEditorTexto as modAp

def saveFileEvent(fileName,text):
    modAp.saveFile(fileName,text)

def readEvent(fileName):
    text=modAp.readText(fileName)
    return text
```

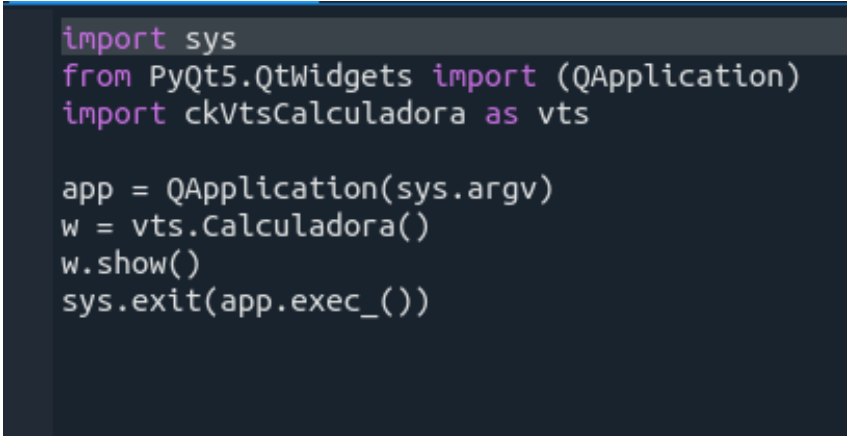
Figura 42: [ckCtrlEditorTexto.py](#)

3.5. Calculadora MVC

En este apartado veremos como hacer una calculadora simple mediante el modelo vista controlador.

3.5.1. ckAppCalculadora

Este fichero actúa como punto de entrada en un modelo Vista-Controlador (MVC). Se encarga de inicializar la aplicación.



```
import sys
from PyQt5.QtWidgets import QApplication
import ckVtsCalculadora as vts

app = QApplication(sys.argv)
w = vts.Calculadora()
w.show()
sys.exit(app.exec_())
```

Figura 43: [ckAppCalculadora.py](#)

3.5.2. ckVtsCalculadora

Este fichero actúa como la Vista. Define la interfaz gráfica y gestiona las interacciones del usuario.

```

from PyQt5.QtWidgets import (QWidget, QPushButton, QLabel, QLineEdit, QVBoxLayout)
import ckControladorCalculadora as ctrl

class Calculadora(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.setWindowTitle('Calculadora MVC')
        self.setGeometry(100, 100, 300, 200)

        self.layout = QVBoxLayout()

        self.num1_input = QLineEdit(self)
        self.num1_input.setPlaceholderText('Ingresa el primer número')
        self.layout.addWidget(self.num1_input)

        self.operador_input = QLineEdit(self)
        self.operador_input.setPlaceholderText('Ingresa el operador (+, -, *, /)')
        self.layout.addWidget(self.operador_input)

        self.num2_input = QLineEdit(self)
        self.num2_input.setPlaceholderText('Ingresa el segundo número')
        self.layout.addWidget(self.num2_input)

        self.boton_calcular = QPushButton('Calcular', self)
        self.boton_calcular.clicked.connect(self.calcular)
        self.layout.addWidget(self.boton_calcular)

        self.resultado_label = QLabel('Resultado: ', self)
        self.layout.addWidget(self.resultado_label)

        self.setLayout(self.layout)
        self.show()

    def mostrar_resultado(self, resultado):
        self.resultado_label.setText(f'Resultado: {resultado}')

    def calcular(self):
        num1 = self.num1_input.text()
        num2 = self.num2_input.text()
        operador = self.operador_input.text()
        resultado = ctrl.calcular(num1, num2, operador)
        self.mostrar_resultado(resultado)

```

Figura 44: [ckVtsCalculadora.py](#)

3.5.3. ckModAppCalculadora

Este fichero actúa como Modelo. Gestiona el cálculo de las operaciones introducidas.

```
def calcular(num1, num2, operador):  
    try:  
        num1, num2 = float(num1), float(num2)  
        if operador == '+':  
            return num1 + num2  
        elif operador == '-':  
            return num1 - num2  
        elif operador == '*':  
            return num1 * num2  
        elif operador == '/':  
            return num1 / num2 if num2 != 0 else 'Error: División por cero'  
        else:  
            return 'Error: Operador no válido'  
    except ValueError:  
        return 'Error: Entrada no válida'
```

Figura 45: [ckModAppCalculadora.py](#)

3.5.4. ckCtrlCalculadora

Este fichero actúa como Controlador. Gestiona los eventos de la aplicación relacionados con los cálculos: llama a las funciones del Modelo (en el módulo ckModAppCalculadora)

```
import ckModAppCalculadora as Mod  
  
def calcular(num1, num2, operador):  
    resultado = Mod.calcular(num1, num2, operador)  
    return resultado
```

Figura 46: [ckCtrlCalculadora.py](#)

4. Parcial 1

4.1. Código desarrollado

4.1.1. Vista

En este código se crea una interfaz gráfica usando PyQt5 para mostrar y gestionar información relacionada con valoraciones de diferentes dominios ("Becas." "Puesto de trabajo"). Dependiendo del dominio seleccionado, carga dinámicamente un módulo externo (como Becas.py) para obtener atributos y criterios, que se presentan en una tabla y una lista. Los valores de los atributos se pueden modificar mediante menús o texto, y al seleccionar un criterio se muestra su descripción. También hay botones para validar los datos(aunque todavía no esta la funcionalidad desarrollada), borrar resultados y cerrar la aplicación.

Enlace de descarga: [Vista.py](#)

```
import sys, importlib
from PyQt5.QtWidgets import (QApplication, QWidget, QGridLayout, QPushButton,
                              QLabel, QComboBox, QTableWidget, QListWidget,
                              QTextEdit, QHeaderView, QHBoxLayout, QTableWidgetItem,
                              QListWidgetItem)
from PyQt5.QtCore import Qt

class VtsExamen(QWidget):
    def __init__(self):
        super().__init__()
        self.Gui()

    def Gui(self):

        #Combobox Dominio
        self.comboboxDominio = QComboBox(self)
        self.dominios = ['Becas', 'Puesto de trabajo']
        self.comboboxDominio.addItem(self.dominios)
        self.comboboxDominio.currentTextChanged.connect(self.actualizarAtributosPorDominio)

        #Combobox Valoracion
        self.comboboxValoracion = QComboBox(self)

        # Mapa de dominios a archivos
        self.dominio_a_modulo = {
            "Becas": "Becas",          # El archivo 'Becas.py'
            "Puesto de trabajo": "Puesto_trabajo" # El archivo 'Puesto_trabajo.py'
        }

        # Verificar si el dominio está en el mapa
        modulo_nombre = self.dominio_a_modulo.get(self.comboboxDominio.currentText())
        if not modulo_nombre:
            return

        # Intentar importar el archivo correspondiente
        try:
            modulo_valoraciones = importlib.import_module(modulo_nombre)
            valoraciones = modulo_valoraciones.valoraciones # Lista de valoraciones
        except (ModuleNotFoundError, AttributeError) as e:
            print(f"Error al cargar valoraciones: {e}")
            return

        # Cargar las valoraciones en el combobox
```

```

self.comboboxValoracion.addItem(valoraciones)

#Creacion tabla
self.tablaCaso = QTableWidgetItem()
self.tablaCaso.setColumnCount(2)
self.tablaCaso.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
self.tablaCaso.setHorizontalHeaderLabels(["ATRIBUTO", "VALOR"])

# Llamamos a la función que devuelve la información de los atributos
caso = modulo_valoraciones.inicializarDatos()

# Definir la cantidad de filas para la tabla
self.tablaCaso.setRowCount(len(caso))

# Rellenar la tabla con los datos de los atributos
for i, (atributo, valor) in enumerate(caso):
    item_atributo = QTableWidgetItem(atributo.nombre) # Nombre del atributo
    item_atributo.setFlags(Qt.ItemIsSelectable | Qt.ItemIsEnabled) # Solo lectura

    # Dependiendo del tipo de atributo, asignamos el valor
    if atributo.tipo == 'multiple' or atributo.tipo == 'bool':
        # Si el atributo tiene opciones múltiples, usamos un ComboBox
        combo_valor = QComboBox()
        combo_valor.addItem(atributo.posiblesValores) # Opciones disponibles
        combo_valor.setCurrentText(str(valor)) # Seleccionar el valor actual
        self.tablaCaso.setCellWidget(i, 1, combo_valor) # Poner el ComboBox en la celda

    else:
        item_valor = QTableWidgetItem(str(valor)) # Convertir el valor a string
        self.tablaCaso.setItem(i, 1, item_valor) # Poner el valor en la tabla

    # Agregar el nombre del atributo
    self.tablaCaso.setItem(i, 0, item_atributo)

#Creacion lista criterios
self.listaCriterios = QListWidget()

#Añadir criterios y conectar descripcion a los criterios
criterios = modulo_valoraciones.inicializarCriterios
    (self.comboboxValoracion.currentText())
for criterio in criterios:
    item = QListWidgetItem(criterio.nombre) # Crea un item para la lista
    item.setData(Qt.UserRole, criterio) # Asocia el objeto Criterio al item
    self.listaCriterios.addItem(item)

self.listaCriterios.currentItemChanged.connect(self.actualizarDescripcion)

#Asociamos el evento al combobox de valoracion una vez definida la
lista, porque si se asocia antes da error
self.comboboxValoracion.currentTextChanged.connect
    (self.actualizarCriteriosPorValoracion)

#Creacion cuadros de texto
self.editorDescripcion = QTextEdit("")
self.editorDescripcion.setReadOnly(True)
self.editorResultado = QTextEdit("")
self.editorResultado.setReadOnly(True)

```

```
self.editorJustificacion = QTextEdit("")
self.editorJustificacion.setReadOnly(True)

#Creacion de labels
self.labelDominio = QLabel("Dominio", self)
self.labelValoracion = QLabel("Valoracion", self)
self.labelCaso = QLabel("Caso", self)
self.labelCriterios = QLabel("Criterios", self)
self.labelDescripcion = QLabel("Descripcion del criterio", self)
self.labelResultado = QLabel("Resultado final", self)
self.labelJustificacion = QLabel("Justificacion de la valoracion", self)

#Creacion de botones
self.btnValidar = QPushButton("Validar", self)
self.btnValidar.resize(self.btnValidar.sizeHint())
self.btnSalir = QPushButton("Salir", self)
self.btnSalir.resize(self.btnSalir.sizeHint())
self.btnBorrar = QPushButton("Borrar", self)
self.btnBorrar.resize(self.btnBorrar.sizeHint())

#Asociar botones con funciones
self.btnSalir.clicked.connect(self.close)
self.btnValidar.clicked.connect(self.validarDatos)
self.btnBorrar.clicked.connect(self.borrar)

#Situvar botones
layout_botones = QHBoxLayout()
layout_botones.addStretch(1)
layout_botones.addWidget(self.btnValidar)
layout_botones.addWidget(self.btnBorrar)
layout_botones.addWidget(self.btnSalir)
layout_botones.addStretch(1)

#Creacion del layout
gridlayout = QGridLayout()

#Posicionar elementos columna 1
gridlayout.addWidget(self.labelDominio, 1, 1, 1, 1)
gridlayout.addWidget(self.comboboxDominio, 2, 1, 1, 5)
gridlayout.addWidget(self.labelValoracion, 3, 1, 1, 1)
gridlayout.addWidget(self.comboboxValoracion, 4, 1, 1, 5)
gridlayout.addWidget(self.labelCaso, 5, 1, 1, 1)
gridlayout.addWidget(self.tablaCaso, 6, 1, 8, 5)
gridlayout.addWidget(self.labelCriterios, 14, 1, 1, 1)
gridlayout.addWidget(self.listaCriterios, 15, 1, 5, 5)
gridlayout.addWidget(self.labelDescripcion, 20, 1, 1, 1)
gridlayout.addWidget(self.editorDescripcion, 21, 1, 8, 5)

#Posicionar elementos columna 2
gridlayout.addWidget(self.labelResultado, 1, 6, 1, 1)
gridlayout.addWidget(self.editorResultado, 2, 6, 12, 6)
gridlayout.addWidget(self.labelJustificacion, 14, 6, 1, 1)
gridlayout.addWidget(self.editorJustificacion, 15, 6, 14, 6)

#Posicionar botones
gridlayout.addLayout(layout_botones, 30, 1, 1, 11)
```

```

self.setLayout(gridlayout)
self.setGeometry(250, 150, 1200, 850)
self.show()

#Funcion asociada al boton de validar datos
def validarDatos(self):
    dominio = self.comboboxDominio.currentText() # Obtener el dominio seleccionado

    if dominio == "Becas":
        resultado = "Ejemplo de resultado para el dominio Becas"
        justificacion = "Ejemplo de justificación para el dominio Becas"
    elif dominio == "Puesto de trabajo":
        resultado = "Ejemplo de resultado para el dominio Puesto de trabajo"
        justificacion = "Ejemplo de justificación para el dominio Puesto de trabajo"
    else:
        resultado = "Error, dominio no reconocido"
        justificacion = "No se ha podido determinar una justificación"

    self.editorResultado.setText(resultado)
    self.editorJustificacion.setText(justificacion)

#Funcion asociada al boton de borrar
def borrar(self):
    self.editorResultado.clear()
    self.editorJustificacion.clear()

#Funcion asociada a la lista de criterios
def actualizarDescripcion(self, current):
    if current:
        # Obtener el criterio seleccionado
        criterio = current.data(Qt.UserRole) # Obtener el objeto Criterio asociado

        # Llamar a la función de descripción del Criterio
        descripcion = criterio.descripcion() if criterio else "Descripción no disponible."

        # Actualizar el QTextEdit con la descripción
        self.editorDescripcion.setText(descripcion)

def actualizarAtributosPorDominio(self, dominio):
    #Eliminamos los valores de la tabla
    self.tablaCaso.clearContents()
    self.tablaCaso.setRowCount(0)

    #Eliminamos los valores del combobox
    self.comboboxValoracion.clear()

    # Verificar si el dominio está en el mapa
    modulo_nombre = self.dominio_a_modulo.get(dominio)
    if not modulo_nombre:
        return

    # Intentar importar el archivo correspondiente
    try:
        modulo_valoraciones = importlib.import_module(modulo_nombre)
        valoraciones = modulo_valoraciones.valoraciones # Lista de valoraciones
    except (ModuleNotFoundError, AttributeError) as e:
        print(f"Error al cargar valoraciones: {e}")

```

```
        return

    # Cargar las valoraciones en el combobox
    self.comboboxValoracion.addItem(valoraciones)

    # Llamamos a la función que devuelve la información de los atributos
    caso = modulo_valoraciones.inicializarDatos()

    # Definir la cantidad de filas para la tabla
    self.tablaCaso.setRowCount(len(caso))

    # Rellenar la tabla con los datos de los atributos
    for i, (atributo, valor) in enumerate(caso):
        item_atributo = QTableWidgetItem(atributo.nombre) # Nombre del atributo
        item_atributo.setFlags(Qt.ItemIsSelectable | Qt.ItemIsEnabled) # Solo lectura

        # Dependiendo del tipo de atributo, asignamos el valor
        if atributo.tipo == 'multiple' or atributo.tipo == 'bool':
            # Si el atributo tiene opciones múltiples, usamos un ComboBox
            combo_valor = QComboBox()
            combo_valor.addItem(atributo.posiblesValores) # Opciones disponibles
            combo_valor.setCurrentText(str(valor)) # Seleccionar el valor actual
            self.tablaCaso.setCellWidget(i, 1, combo_valor) # Poner el ComboBox en la celda

        else:
            item_valor = QTableWidgetItem(str(valor)) # Convertir el valor a string
            self.tablaCaso.setItem(i, 1, item_valor) # Poner el valor en la tabla

        # Agregar el nombre del atributo
        self.tablaCaso.setItem(i, 0, item_atributo)

def actualizarCriteriosPorValoracion(self, valoracion):
    # Limpiar la lista de criterios
    self.listaCriterios.clear()
    self.editorDescripcion.clear()

    # Verificar si el dominio está en el mapa
    modulo_nombre = self.dominio_a_modulo.get(self.comboboxDominio.currentText())
    if not modulo_nombre:
        return

    # Intentar importar el módulo correspondiente a la valoración seleccionada
    try:
        modulo_valoraciones = importlib.import_module(modulo_nombre)

        # Cargar los criterios para la valoración seleccionada
        criterios = modulo_valoraciones.inicializarCriterios(valoracion)

        # Añadir criterios a la lista
        for criterio in criterios:
            # Crear un ítem en la lista con el nombre del criterio
            item = QListWidgetItem(criterio.nombre)

            # Asocia el objeto Criterio completo al ítem de la lista usando setData
            item.setData(Qt.UserRole, criterio) # Almacena el objeto Criterio con UserRole
```

```
        # Añadir el ítem a la lista de criterios
        self.listaCriterios.addItem(item)

    except (ModuleNotFoundError, AttributeError) as e:
        print(f"Error al cargar los criterios: {e}")
        return

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = VtsExamen()
    ex.show()
    sys.exit(app.exec_())
```

4.1.2. Becas

En este código se define la base de conocimiento para representar atributos y criterios utilizados en un sistema de validación de solicitudes de becas. Los atributos (como edad, nota o nivel de inglés) se modelan con clases específicas, y los criterios determinan cómo se evalúan esos atributos según reglas predefinidas. A través de la función "inicializarDatos()" se crean ejemplos de datos, y con "inicializarCriterios()" se generan listas de criterios según el tipo de valoración ("General." "MEC"). Cada criterio aporta una puntuación y puede ser obligatorio o no.

Enlace de descarga: [Becas.py](#)

```
valoraciones = ['General','MEC']

class Atributo():
    '''Especificamos las propiedades de los atributos que van a usarse
    en la base de conocimiento para describir un caso'''
    def __init__(self,nombre,tipo,posiblesValores=None):
        self.nombre = nombre #nombre
        self.tipo = tipo #tipo de atributo
        if(tipo == 'bool'): #BOOLEANOS
            self.posiblesValores = ['True','False']
        elif(tipo == 'multiple'): #Multiple eleccion
            self.posiblesValores = posiblesValores

class Edad(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Edad','int', None)
class Ingles(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Nivel de ingles','multiple',['Ninguno','B1','B2','C1','C2'])
class Dinero(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Patrimonio familiar','int',None)
class Nota(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Nota media','int',None)
class Hermanos(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Tiene hermanos','bool',None)

def inicializarDatos():
    edad=(Edad(),20)
    dinero=(Dinero(),10000)
    notaMedia=(Nota(), 9)
    nivelIngles=(Ingles(), 'B2')
    hermanos=(Hermanos(), False)
    solicitud=(edad,dinero,notaMedia,nivelIngles,hermanos))
    return solicitud

class Criterio():
    '''Establece un criterio a evaluar'''
    def __init__(self,nombre,atributo,tipoComparacion,tipoResultado,
        puntuacion,valor,obligatoria):
        self.nombre = nombre #nombre del criterio
        self.atributo = atributo #atributo
        self.tipoComparacion = tipoComparacion #igual, mayor, menor, rango
```

```

self.tipoResultado = tipoResultado #unidad del atributo
self.puntuacion = puntuacion #los puntos que da al ser aceptado
self.valor = valor #valor que debe tener el atributo para que se cumpla el criterio.
# a puede ser una tupla para una comparación multiple
self.obligatoria = obligatoria #si la condicion tiene que ser cumplida obligatoriamente
def descripcion(self):
    '''Describimos el criterio en el panel del programa'''
    self.desc = u'Nombre:\t\t' + self.nombre + u'\nComparación:\t' + self.tipoComparacion
    self.desc+= u'\nTipo de resultado:\t'+str(self.tipoResultado)
    self.desc += u'\nPuntuación:\t' + str(self.puntuacion) + u'\nValor:\t\t'
    self.desc += str(self.valor) + u'\nTerminal:\t\t' + str(self.obligatoria)
    return self.desc

def inicializarCriterios(valoracion):
    criterios = []
    if valoracion == 'General':
        crt1=Criterio('Edad', Edad(), 'rango', 'año', 10, [16,26], True)
        crt2=Criterio('Nota media[5-7]', Nota(), 'rango', 'puntos', 20, [5,7], False)
        crt3=Criterio('Nota media[7-8]', Nota(), 'rango', 'puntos', 30, [7,8], False)
        crt4=Criterio('Nota media[8-10]', Nota(), 'rango', 'puntos', 50, [8,10], False)
        crt5=Criterio('Nivel de ingles-Ninguno', Ingles(), 'categorica',
            None, 0, ['Ninguno'],False)
        crt6=Criterio('Nivel de ingles-B1', Ingles(), 'categorica', None, 5, ['B1'],False)
        crt7=Criterio('Nivel de ingles-B2', Ingles(), 'categorica', None, 10, ['B2'],False)
        crt8=Criterio('Nivel de ingles-C1', Ingles(), 'categorica', None, 20, ['C1'],False)
        crt9=Criterio('Nivel de ingles-C2', Ingles(), 'categorica', None, 30, ['C2'],False)
        crt10=Criterio('Patrimonio familiar', Dinero(), 'rango', 'euros', 30, [0,80000], True)

        criterios.append(crt1)
        criterios.append(crt2)
        criterios.append(crt3)
        criterios.append(crt4)
        criterios.append(crt5)
        criterios.append(crt6)
        criterios.append(crt7)
        criterios.append(crt8)
        criterios.append(crt9)
        criterios.append(crt10)

    elif valoracion == 'MEC':
        crt1=Criterio('Edad', Edad(), 'rango', 'año', 10, [16,26], True)
        crt2=Criterio('Nota media[5-7]', Nota(), 'rango', 'puntos', 20, [5,7], False)
        crt3=Criterio('Nota media[7-8]', Nota(), 'rango', 'puntos', 30, [7,8], False)
        crt4=Criterio('Nota media[8-10]', Nota(), 'rango', 'puntos', 50, [8,10], False)
        crt5=Criterio('Nivel de ingles-Ninguno', Ingles(), 'categorica',
            None, 0, ['Ninguno'],False)
        crt6=Criterio('Nivel de ingles-B1', Ingles(), 'categorica', None, 0, ['B1'],False)
        crt7=Criterio('Nivel de ingles-B2', Ingles(), 'categorica', None, 10, ['B2'],False)
        crt8=Criterio('Nivel de ingles-C1', Ingles(), 'categorica', None, 15, ['C1'],False)
        crt9=Criterio('Nivel de ingles-C2', Ingles(), 'categorica', None, 20, ['C2'],False)
        crt10=Criterio('Patrimonio familiar', Dinero(), 'rango', 'euros', 30, [0,100000], True)

        criterios.append(crt1)
        criterios.append(crt2)
        criterios.append(crt3)

```



```
    criterios.append(crt4)
    criterios.append(crt5)
    criterios.append(crt6)
    criterios.append(crt7)
    criterios.append(crt8)
    criterios.append(crt9)
    criterios.append(crt10)

    return criterios
```

4.1.3. Puesto de trabajo

En este código se modela la base de conocimiento para representar atributos y criterios utilizados en un sistema de validación para solicitudes laborales, utilizando atributos como edad, experiencia, titulación e inglés. Se definen clases para representar estos atributos y una clase Criterio que establece condiciones y puntuaciones para evaluar dichos atributos. Según el tipo de valoración (“Secretario” o “Gerente”), la función “inicializarCriterios()” devuelve una lista de criterios específicos con sus respectivas reglas y puntuaciones. Además, se generan unos datos de ejemplo con la función “inicializarDatos()”.

Enlace de descarga: [Puesto_trabajo.py](#)

```
valoraciones = ['Secretario','Gerente']

class Atributo():
    '''Especificamos las propiedades de los atributos que van a usarse
    en la base de conocimiento para describir un caso'''
    def __init__(self,nombre,tipo,posiblesValores=None):
        self.nombre = nombre #nombre
        self.tipo = tipo #tipo de atributo
        if(tipo == 'bool'): #BOOLEANOS
            self.posiblesValores = ['True','False']
        elif(tipo == 'multiple'): #Multiple eleccion
            self.posiblesValores = posiblesValores

class Edad(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Edad','int',None)
class Ingles(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Nivel de ingles','multiple',['Ninguno','B1','B2','C1','C2'])
class Titulacion(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Ultimo titulacion','multiple',
            ['ESO','Bachiller','Grado Medio','Grado Superior','Grado Universitario','Master'])
class Experiencia(Atributo):
    def __init__(self):
        Atributo.__init__(self,'Años de experiencia','int',None)

def inicializarDatos():
    edad=(Edad(),20)
    titulacion=(Titulacion(), 'ESO')
    experiencia=(Experiencia(), 3)
    nivelIngles=(Ingles(), 'B2')
    solicitud=(edad,titulacion,experiencia,nivelIngles])
    return solicitud

class Criterio():
    '''Establece un criterio a evaluar'''
    def __init__(self,nombre,atributo,tipoComparacion,tipoResultado,
        puntuacion,valor,obligatoria):
        self.nombre = nombre #nombre del criterio
        self.atributo = atributo #atributo
        self.tipoComparacion = tipoComparacion #igual, mayor, menor, rango
        self.tipoResultado = tipoResultado #unidad del atributo
        self.puntuacion = puntuacion #los puntos que da al ser aceptado
        self.valor = valor #valor que debe tener el atributo para que se cumpla el criterio.
        #a puede ser una tupla para una comparación multiple
```

```

        self.obligatoria = obligatoria #si la condicion tiene que ser cumplida obligatoriamente
def descripcion(self):
    '''Describimos el criterio en el panel del programa'''
    self.desc = u'Nombre:\t\t' + self.nombre + u'\nComparación:\t' + self.tipoComparacion
    self.desc+= u'\nTipo de resultado:\t'+str(self.tipoResultado)
    self.desc += u'\nPuntuación:\t' + str(self.puntuacion) + u'\nValor:\t\t'
    self.desc += str(self.valor) + u'\nTerminal:\t\t' + str(self.obligatoria)
    return self.desc

def inicializarCriterios(valoracion):
    criterios = []
    if valoracion == 'Secretario':
        crt1=Criterio('Edad', Edad(), 'rango', 'año', 10, [18,50], True)
        crt2=Criterio('Nivel de ingles - Ninguno', Ingles(), 'categorica',
            None, 0, ['Ninguno'],False)
        crt3=Criterio('Nivel de ingles - B1', Ingles(), 'categorica', None, 5, ['B1'],False)
        crt4=Criterio('Nivel de ingles - B2', Ingles(), 'categorica', None, 10, ['B2'],False)
        crt5=Criterio('Nivel de ingles - C1', Ingles(), 'categorica', None, 20, ['C1'],False)
        crt6=Criterio('Nivel de ingles - C2', Ingles(), 'categorica', None, 30, ['C2'],False)
        crt7=Criterio('Experiencia laboral', Experiencia(), 'rango', 'año', 30, [3,60], True)
        crt8=Criterio('Titulación - ESO', Titulacion(), 'categorica',
            None, 10, ['ESO'], False)
        crt9=Criterio('Titulación - Bachiller', Titulacion(), 'categorica',
            None, 20, ['Bachiller'], False)
        crt10=Criterio('Titulación - Grado Medio', Titulacion(), 'categorica',
            None, 15, ['Grado Medio'], False)
        crt11=Criterio('Titulación - Grado Superior', Titulacion(), 'categorica',
            None, 25, ['Grado Superior'], False)
        crt12=Criterio('Titulación - Grado Universitario', Titulacion(), 'categorica',
            None, 40, ['Grado Universitario'], False)
        crt13=Criterio('Titulación - Master', Titulacion(), 'categorica',
            None, 50, ['Master'], False)

        criterios.append(crt1)
        criterios.append(crt2)
        criterios.append(crt3)
        criterios.append(crt4)
        criterios.append(crt5)
        criterios.append(crt6)
        criterios.append(crt7)
        criterios.append(crt8)
        criterios.append(crt9)
        criterios.append(crt10)
        criterios.append(crt11)
        criterios.append(crt12)
        criterios.append(crt13)

    elif valoracion == 'Gerente':
        crt1=Criterio('Edad', Edad(), 'rango', 'año', 10, [20,48], True)
        crt2=Criterio('Nivel de ingles-Ninguno', Ingles(), 'categorica',
            None, 0, ['Ninguno'],False)
        crt3=Criterio('Nivel de ingles-B1', Ingles(), 'categorica', None, 0, ['B1'],False)
        crt4=Criterio('Nivel de ingles-B2', Ingles(), 'categorica', None, 10, ['B2'],False)
        crt5=Criterio('Nivel de ingles-C1', Ingles(), 'categorica', None, 15, ['C1'],False)
        crt6=Criterio('Nivel de ingles-C2', Ingles(), 'categorica', None, 20, ['C2'],False)
        crt7=Criterio('Experiencia laboral', Experiencia(), 'rango', 'año', 30, [5,60], True)
        crt8=Criterio('Titulación - ESO', Titulacion(), 'categorica', None, 0, ['ESO'], False)

```

```
crt9=Criterio('Titulación - Bachiller', Titulacion(), 'categorica',
             None, 10, ['Bachiller'], False)
crt10=Criterio('Titulación - Grado Medio', Titulacion(), 'categorica',
              None, 5, ['Grado Medio'], False)
crt11=Criterio('Titulación - Grado Superior', Titulacion(), 'categorica',
              None, 15, ['Grado Superior'], False)
crt12=Criterio('Titulación - Grado Universitario', Titulacion(), 'categorica',
              None, 30, ['Grado Universitario'], False)
crt13=Criterio('Titulación - Master', Titulacion(), 'categorica',
              None, 60, ['Master'], False)

criterios.append(crt1)
criterios.append(crt2)
criterios.append(crt3)
criterios.append(crt4)
criterios.append(crt5)
criterios.append(crt6)
criterios.append(crt7)
criterios.append(crt8)
criterios.append(crt9)
criterios.append(crt10)
criterios.append(crt11)
criterios.append(crt12)
criterios.append(crt13)

return criterios
```

4.1.4. Imagen de la vista creada

The screenshot shows a window titled "Vista.py" with a light gray border and standard window controls. The interface is divided into several sections:

- Domain and General Settings:** At the top left, there are two dropdown menus. The first is labeled "Dominio" and has "Becas" selected. The second is labeled "Valoracion" and has "General" selected.
- Case Table:** Below the dropdowns is a section labeled "Caso". It contains a table with two columns: "ATRIBUTO" and "VALOR".

	ATRIBUTO	VALOR
1	Edad	20
2	Patrimonio familiar	10000
3	Nota media	9
4	Nivel de ingles	B2
5	Tiene hermanos	False
- Criteria List:** Below the table is a section labeled "Criterios". It contains a list of criteria: "Edad", "Nota media[5-7]", "Nota media[7-8]", "Nota media[8-10]", "Nivel de ingles-Ninguno", "Nivel de ingles-B1", "Nivel de ingles-B2", "Nivel de ingles-C1", "Nivel de ingles-C2", and "Patrimonio familiar". The "Edad" criterion is currently selected and highlighted in blue.
- Criterion Description:** Below the list is a section labeled "Descripcion del criterio". It contains a table with two columns: "Nombre:" and "Valor:". The "Nombre:" column contains "Edad", "Comparación:", "Tipo de resultado:", "Puntuación: 10", "Valor:", and "Terminal:". The "Valor:" column contains "rango", "año", "[16, 26]", and "True".
- Result and Justification:** On the right side of the window, there are two large text areas. The top one is labeled "Resultado final" and contains the text "Ejemplo de resultado para el dominio Becas". The bottom one is labeled "Justificacion de la valoracion" and contains the text "Ejemplo de justificación para el dominio Becas".
- Buttons:** At the bottom of the window, there are three buttons: "Validar", "Borrar", and "Salir".

Figura 47: Vista.py

4.2. Video explicativo

Enlace al video: <https://youtu.be/SPoH0TCEHUK>

5. Practica 4

5.1. Resumen de mi experiencia realizando el tutorial

Al comenzar a seguir este tutorial, debo admitir que me sentía bastante perdido. Nunca antes había utilizado la herramienta Protégé, y eso ya suponía un desafío inicial. A esto se sumaba el hecho de que el tutorial estaba basado en la versión 4.3 del programa, mientras que yo estaba trabajando con la versión 5.6. Esta diferencia entre versiones generó algunas complicaciones, sobre todo al principio, ya que varias de las opciones que se indicaban en el tutorial no se encontraban en el mismo lugar o incluso habían cambiado de nombre. En algunos casos, funcionalidades que antes estaban separadas habían sido agrupadas bajo una misma categoría o menú, lo cual me obligó a explorar un poco por mi cuenta para poder ubicarme.

A pesar de esta dificultad inicial, una vez que logré identificar dónde se encontraban las opciones mencionadas y me familiaricé con la interfaz, el resto del proceso resultó mucho más fluido. La herramienta Protégé, en general, me pareció bastante intuitiva y amigable para el usuario. Además, el tutorial estaba muy bien estructurado, con explicaciones claras y detalladas paso a paso, lo que facilitó enormemente la comprensión de los conceptos y tareas. En definitiva, aunque el comienzo fue un poco confuso, la experiencia terminó siendo bastante positiva y enriquecedora.

[Haz click aqui para descargar el archivo Pizza.rdf](#)

5.2. Lenguajes de marcas

En la práctica se trabajó principalmente con los lenguajes RDF y OWL, los cuales desempeñan un papel fundamental en la representación estructurada de información y conocimiento dentro de un dominio específico.

5.2.1. RDF

RDF (Resource Description Framework) es un lenguaje de marcado diseñado para describir recursos y las relaciones entre ellos, utilizando una estructura de sujeto-predicado-objeto. Su principal objetivo es proporcionar una forma estándar de representar información semántica en la web, permitiendo que los datos puedan ser entendidos y procesados por máquinas.

5.2.2. OWL

OWL (Web Ontology Language), por su parte, se construye sobre RDF y añade una capa adicional de expresividad. Está diseñado para representar conocimiento complejo sobre categorías, propiedades y relaciones entre conceptos dentro de un dominio. OWL permite definir restricciones, jerarquías y axiomas lógicos, lo que resulta muy útil para aplicaciones que requieren una representación más rica y precisa del conocimiento, como los sistemas de razonamiento o inferencia automática.

5.2.3. Turtle

En cuanto a Turtle, aunque no apareció directamente aplicado en la práctica, sí fue explicado previamente en clase, por lo que considero relevante mencionarlo. Turtle (Terse RDF Triple Language) es un formato de serialización para expresar datos RDF de forma más legible y compacta en comparación con otras sintaxis como RDF/XML. Su uso facilita la escritura y comprensión de los triples RDF, especialmente en entornos educativos o de desarrollo. Si bien su papel es más de representación sintáctica que semántica, resulta clave para facilitar el trabajo con datos RDF y mejorar la claridad en la edición manual de ontologías.

En conjunto, estos lenguajes conforman una base sólida para la representación del conocimiento en sistemas basados en la web semántica, permitiendo modelar, compartir e intercambiar información de forma estructurada, reutilizable y comprensible tanto por humanos como por máquinas.

6. Practica 5

En esta práctica se han explorado herramientas que permiten integrar y aplicar los conocimientos adquiridos en prácticas anteriores. En concreto, nos hemos centrado en el uso de bibliotecas de Python que facilitan la lectura, manipulación y análisis de archivos en formato OWL (Web Ontology Language), un estándar utilizado para representar información estructurada y relaciones entre conceptos dentro de una ontología.

Para llevar a cabo esta tarea, hemos empleado dos librerías ampliamente utilizadas en el ámbito del procesamiento de ontologías: `owlready2` y `rdflib`.

- La librería `owlready2` nos ha permitido cargar archivos OWL y trabajar con ellos de forma sencilla, mediante una interfaz orientada a objetos que facilita la consulta y modificación de clases, propiedades y relaciones definidas en la ontología.
- Por otro lado, `rdflib` nos ha proporcionado una mayor flexibilidad al momento de realizar consultas más complejas sobre los datos RDF subyacentes, permitiéndonos utilizar el lenguaje de consultas SPARQL para extraer información específica de las ontologías.

El uso de estas librerías no solo ha sido útil en el contexto de esta práctica, sino que también será fundamental más adelante en el desarrollo del Parcial 2, donde se requerirá aplicar estos conocimientos para resolver problemas más avanzados relacionados con el manejo y la consulta de ontologías.

Gracias al uso combinado de estas herramientas, hemos podido comprender mejor cómo acceder, consultar y manipular información semántica contenida en ontologías, lo cual resulta esencial para el desarrollo de aplicaciones que requieran razonamiento semántico o integración de conocimiento estructurado.

7. Parcial 2

7.1. Archivos OWL

En este apartado se definen los archivos OWL desarrollados para el examen, entre estos tenemos 3, "esq_dom.owl" que se encarga de cargar el esquema del dominio de la aplicación, es decir la terminología general, además de este tenemos a "Becas.owl" y "Puesto_trabajo.owl" que son los archivos que contienen la información de las becas y puestos de trabajo respectivamente, es decir las bases del conocimiento de la aplicación.

7.1.1. Esq_dom

Este fichero define una ontología en OWL para modelar un sistema de valoración. Incluye tres clases principales: Atributo, Criterio y Valoracion. Los Atributos tienen propiedades como nombre, tipo, posiblesValores y valorInicial. Los Criterios describen cómo evaluar esos atributos mediante propiedades como tipoComparacion, valor, puntuacion y obligatoria. Además, se establece una relación entre Criterio y Valoracion mediante una propiedad de objeto. En conjunto, el modelo permite representar de forma estructurada reglas y valoraciones aplicadas a atributos.

Enlace de descarga: [Esq_dom.owl](#)

```
<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/esq_dom#"
  xml:base="http://example.org/esq_dom"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">

  <owl:Ontology rdf:about="http://example.org/esq_dom"/>

  <!-- Clases para Atributo, Criterio y Valoracion -->
  <owl:Class rdf:about="#Atributo"/>
  <owl:Class rdf:about="#Criterio"/>
  <owl:Class rdf:about="#Valoracion"/>

  <!-- Propiedades de datos (para Atributos) -->
  <owl:DatatypeProperty rdf:about="#nombre">
    <rdfs:domain rdf:resource="#Atributo"/>
    <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty rdf:about="#tipo">
    <rdfs:domain rdf:resource="#Atributo"/>
    <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty rdf:about="#posiblesValores">
    <rdfs:domain rdf:resource="#Atributo"/>
    <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty rdf:about="#valorInicial">
    <rdfs:domain rdf:resource="#Atributo"/>
    <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
  </owl:DatatypeProperty>
```

```
<!-- Propiedades de datos (para Criterios) -->
<owl:DatatypeProperty rdf:about="#tipoComparacion">
  <rdfs:domain rdf:resource="#Criterio"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="#tipoResultado">
  <rdfs:domain rdf:resource="#Criterio"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="#puntuacion">
  <rdfs:domain rdf:resource="#Criterio"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="#valor">
  <rdfs:domain rdf:resource="#Criterio"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="#obligatoria">
  <rdfs:domain rdf:resource="#Criterio"/>
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#boolean"/>
</owl:DatatypeProperty>

<!-- Propiedad de conexión entre Criterio y Valoración -->
<owl:ObjectProperty rdf:about="#valoracion">
  <rdfs:domain rdf:resource="#Criterio"/>
  <rdfs:range rdf:resource="#Valoracion"/>
</owl:ObjectProperty>

</rdf:RDF>
```

7.1.2. Becas

Este documento OWL define instancias para un sistema de evaluación de becas basado en una ontología externa (esq_dom). Incluye dos tipos de becas (General y MEC) modeladas como instancias de Valoracion. Cada beca contiene múltiples Criterios, que evalúan distintos Atributos como edad, nota media, nivel de inglés o patrimonio familiar. Estos criterios especifican el tipo de comparación (rango o categórica), los valores esperados, la puntuación asignada y si son obligatorios.

Enlace de descarga: [Becas.owl](#)

```
<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/becas#"
  xml:base="http://example.org/becas"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:esq_dom="http://example.org/esq_dom#">

  <owl:Ontology rdf:about="http://example.org/becas">
    <owl:imports rdf:resource="http://example.org/esq_dom"/>
  </owl:Ontology>

  <!-- Instancias de Valoraciones -->
  <owl:NamedIndividual rdf:about="#General">
    <rdf:type rdf:resource="http://example.org/esq_dom#Valoracion"/>
    <esq_dom:nombre>General</esq_dom:nombre>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#MEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Valoracion"/>
    <esq_dom:nombre>MEC</esq_dom:nombre>
  </owl:NamedIndividual>

  <!-- Instancias de Atributos -->
  <owl:NamedIndividual rdf:about="#Edad">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Edad</esq_dom:nombre>
    <esq_dom:tipo>int</esq_dom:tipo>
    <esq_dom:valorInicial>20</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Ingles">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Nivel de ingles</esq_dom:nombre>
    <esq_dom:tipo>multiple</esq_dom:tipo>
    <esq_dom:posiblesValores>Ninguno</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>B1</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>B2</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>C1</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>C2</esq_dom:posiblesValores>
    <esq_dom:valorInicial>Ninguno</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Dinero">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Patrimonio familiar</esq_dom:nombre>
    <esq_dom:tipo>int</esq_dom:tipo>
```

```

    <esq_dom:valorInicial>10000</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Nota">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Nota media</esq_dom:nombre>
    <esq_dom:tipo>int</esq_dom:tipo>
    <esq_dom:valorInicial>8</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Hermanos">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Tiene hermanos</esq_dom:nombre>
    <esq_dom:tipo>bool</esq_dom:tipo>
    <esq_dom:valorInicial>True</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <!-- Instancias de Criterios Beca General -->
  <owl:NamedIndividual rdf:about="#CriterioEdadGeneral">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Edad</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Edad"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
    <esq_dom:puntuacion>10</esq_dom:puntuacion>
    <esq_dom:valor>16,26</esq_dom:valor>
    <esq_dom:obligatoria>true</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#General"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNotaMedia[5-7]General">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nota media[5-7]</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Nota"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>puntos</esq_dom:tipoResultado>
    <esq_dom:puntuacion>20</esq_dom:puntuacion>
    <esq_dom:valor>[5,7]</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#General"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNotaMedia[7-8]General">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nota media[7-8]</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Nota"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>puntos</esq_dom:tipoResultado>
    <esq_dom:puntuacion>30</esq_dom:puntuacion>
    <esq_dom:valor>[7,8]</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#General"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNotaMedia[8-10]General">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nota media[8-10]</esq_dom:nombre>

```

```

    <esq_dom:atributo rdf:resource="#Nota"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>puntos</esq_dom:tipoResultado>
    <esq_dom:puntuacion>50</esq_dom:puntuacion>
    <esq_dom:valor>[8,10]</esq_dom:valor>
    <esq_dom:obligatoria>>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioNivelIngles-NingunoGeneral">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-Ninguno</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>0</esq_dom:puntuacion>
  <esq_dom:valor>Ninguno</esq_dom:valor>
  <esq_dom:obligatoria>>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioNivelIngles-B1General">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-B1</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>5</esq_dom:puntuacion>
  <esq_dom:valor>B1</esq_dom:valor>
  <esq_dom:obligatoria>>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioNivelIngles-B2General">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-B2</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>10</esq_dom:puntuacion>
  <esq_dom:valor>B2</esq_dom:valor>
  <esq_dom:obligatoria>>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioNivelIngles-C1General">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-C1</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>20</esq_dom:puntuacion>
  <esq_dom:valor>C1</esq_dom:valor>
  <esq_dom:obligatoria>>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

```

```

<owl:NamedIndividual rdf:about="#CriterioNivelIngles-C2General">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-C2</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>30</esq_dom:puntuacion>
  <esq_dom:valor>C2</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioPatrimonioFamiliarGeneral">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Patrimonio familiar</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Dinero"/>
  <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado>euros</esq_dom:tipoResultado>
  <esq_dom:puntuacion>30</esq_dom:puntuacion>
  <esq_dom:valor>[0,80000]</esq_dom:valor>
  <esq_dom:obligatoria>true</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#General"/>
</owl:NamedIndividual>

<!-- Instancias de Criterios Beca MEC -->
<owl:NamedIndividual rdf:about="#CriterioEdadMEC">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Edad</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Edad"/>
  <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
  <esq_dom:puntuacion>10</esq_dom:puntuacion>
  <esq_dom:valor>16,26</esq_dom:valor>
  <esq_dom:obligatoria>true</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#MEC"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioNotaMedia[5-7]MEC">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nota media[5-7]</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Nota"/>
  <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado>puntos</esq_dom:tipoResultado>
  <esq_dom:puntuacion>20</esq_dom:puntuacion>
  <esq_dom:valor>[5,7]</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#MEC"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioNotaMedia[7-8]MEC">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nota media[7-8]</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Nota"/>
  <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado>puntos</esq_dom:tipoResultado>

```

```

    <esq_dom:puntuacion>30</esq_dom:puntuacion>
    <esq_dom:valor>[7,8]</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNotaMedia[8-10]MEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nota media[8-10]</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Nota"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>puntos</esq_dom:tipoResultado>
    <esq_dom:puntuacion>50</esq_dom:puntuacion>
    <esq_dom:valor>[8,10]</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNivelIngles-NingunoMEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-Ninguno</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>0</esq_dom:puntuacion>
    <esq_dom:valor>Ninguno</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNivelIngles-B1MEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-B1</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>0</esq_dom:puntuacion>
    <esq_dom:valor>B1</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNivelIngles-B2MEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-B2</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>10</esq_dom:puntuacion>
    <esq_dom:valor>B2</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNivelIngles-C1MEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-C1</esq_dom:nombre>

```



```

    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>15</esq_dom:puntuacion>
    <esq_dom:valor>C1</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioNivelIngles-C2MEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-C2</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>20</esq_dom:puntuacion>
    <esq_dom:valor>C2</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioPatrimonioFamiliarMEC">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Patrimonio familiar</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Dinero"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>euros</esq_dom:tipoResultado>
    <esq_dom:puntuacion>30</esq_dom:puntuacion>
    <esq_dom:valor>[0,80000]</esq_dom:valor>
    <esq_dom:obligatoria>true</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#MEC"/>
  </owl:NamedIndividual>

</rdf:RDF>

```

7.1.3. Puesto_trabajo

Este fichero OWL describe instancias relacionadas con un modelo de evaluación de puestos de trabajo, utilizando una ontología externa (esq_dom). En él se definen dos tipos principales de elementos: valoraciones y atributos. Las valoraciones representan posibles evaluaciones de un puesto, como “Secretario” y “Gerente”. Por otro lado, los atributos describen características que pueden asociarse a un perfil laboral, como la edad, el nivel de inglés, la titulación académica o los años de experiencia. Cada atributo incluye información sobre su tipo (por ejemplo, entero o múltiple), un valor inicial, y en algunos casos una lista de posibles valores.

Enlace de descarga: [Puesto_trabajo.owl](#)

```
<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/puestoTrabajo#"
  xml:base="http://example.org/puestoTrabajo"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:esq_dom="http://example.org/esq_dom#">

  <owl:Ontology rdf:about="http://example.org/puestoTrabajo">
    <owl:imports rdf:resource="http://example.org/esq_dom"/>
  </owl:Ontology>

  <!-- Instancias de Valoraciones -->
  <owl:NamedIndividual rdf:about="#Secretario">
    <rdf:type rdf:resource="http://example.org/esq_dom#Valoracion"/>
    <esq_dom:nombre>Secretario</esq_dom:nombre>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Gerente">
    <rdf:type rdf:resource="http://example.org/esq_dom#Valoracion"/>
    <esq_dom:nombre>Gerente</esq_dom:nombre>
  </owl:NamedIndividual>

  <!-- Instancias de Atributos -->
  <owl:NamedIndividual rdf:about="#Edad">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Edad</esq_dom:nombre>
    <esq_dom:tipo>int</esq_dom:tipo>
    <esq_dom:valorInicial>20</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Ingles">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Nivel de ingles</esq_dom:nombre>
    <esq_dom:tipo>multiples</esq_dom:tipo>
    <esq_dom:posiblesValores>Ninguno</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>B1</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>B2</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>C1</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>C2</esq_dom:posiblesValores>
    <esq_dom:valorInicial>Ninguno</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Titulacion">
```

```

    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Mayor titulación académica</esq_dom:nombre>
    <esq_dom:tipo>multiple</esq_dom:tipo>
    <esq_dom:posiblesValores>ESO</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>Bachiller</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>Grado Medio</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>Grado Superior</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>Grado Universitario</esq_dom:posiblesValores>
    <esq_dom:posiblesValores>Master</esq_dom:posiblesValores>
    <esq_dom:valorInicial>ESO</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#Experiencia">
    <rdf:type rdf:resource="http://example.org/esq_dom#Atributo"/>
    <esq_dom:nombre>Años de experiencia</esq_dom:nombre>
    <esq_dom:tipo>int</esq_dom:tipo>
    <esq_dom:valorInicial>3</esq_dom:valorInicial>
  </owl:NamedIndividual>

  <!-- Instancias de Criterios Puesto de Secretario -->
  <owl:NamedIndividual rdf:about="#CriterioEdadSecretario">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Edad</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Edad"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
    <esq_dom:puntuacion>10</esq_dom:puntuacion>
    <esq_dom:valor>[18,50]</esq_dom:valor>
    <esq_dom:obligatoria>true</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Secretario"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioInglesNingunoSecretario">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles - Ninguno</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>0</esq_dom:puntuacion>
    <esq_dom:valor>Ninguno</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Secretario"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioInglesB1Secretario">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles - B1</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>5</esq_dom:puntuacion>
    <esq_dom:valor>B1</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Secretario"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioInglesB2Secretario">

```

```

<rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
<esq_dom:nombre>Nivel de ingles - B2</esq_dom:nombre>
<esq_dom:atributo rdf:resource="#Ingles"/>
<esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
<esq_dom:tipoResultado></esq_dom:tipoResultado>
<esq_dom:puntuacion>10</esq_dom:puntuacion>
<esq_dom:valor>B2</esq_dom:valor>
<esq_dom:obligatoria>false</esq_dom:obligatoria>
<esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioInglesC1Secretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles - C1</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>20</esq_dom:puntuacion>
  <esq_dom:valor>C1</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioInglesC2Secretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles - C2</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>30</esq_dom:puntuacion>
  <esq_dom:valor>C2</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioExperienciaSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Experiencia laboral</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Experiencia"/>
  <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
  <esq_dom:puntuacion>30</esq_dom:puntuacion>
  <esq_dom:valor>[3,60]</esq_dom:valor>
  <esq_dom:obligatoria>true</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionESOSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - ESO</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>10</esq_dom:puntuacion>
  <esq_dom:valor>ESO</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>

```

```

</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionBachillerSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Bachiller</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>20</esq_dom:puntuacion>
  <esq_dom:valor>Bachiller</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionGradoMedioSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Grado Medio</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>15</esq_dom:puntuacion>
  <esq_dom:valor>Grado Medio</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionGradoSuperiorSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Grado Superior</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>25</esq_dom:puntuacion>
  <esq_dom:valor>Grado Superior</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionUniversitariaSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Grado Universitario</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>40</esq_dom:puntuacion>
  <esq_dom:valor>Grado Universitario</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Secretario"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionMasterSecretario">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Master</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>50</esq_dom:puntuacion>

```

```

    <esq_dom:valor>Master</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Secretario"/>
  </owl:NamedIndividual>

<!-- Instancias de Criterios Puesto de Gerente -->
  <owl:NamedIndividual rdf:about="#CriterioEdadGerente">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Edad</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Edad"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
    <esq_dom:puntuacion>10</esq_dom:puntuacion>
    <esq_dom:valor>[20,48]</esq_dom:valor>
    <esq_dom:obligatoria>true</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Gerente"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioEdadGerente">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Edad</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Edad"/>
    <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
    <esq_dom:puntuacion>10</esq_dom:puntuacion>
    <esq_dom:valor>[20,48]</esq_dom:valor>
    <esq_dom:obligatoria>true</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Gerente"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioInglesNingunoGerente">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-Ninguno</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>0</esq_dom:puntuacion>
    <esq_dom:valor>Ninguno</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Gerente"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioInglesB1Gerente">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
    <esq_dom:nombre>Nivel de ingles-B1</esq_dom:nombre>
    <esq_dom:atributo rdf:resource="#Ingles"/>
    <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
    <esq_dom:tipoResultado></esq_dom:tipoResultado>
    <esq_dom:puntuacion>0</esq_dom:puntuacion>
    <esq_dom:valor>B1</esq_dom:valor>
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Gerente"/>
  </owl:NamedIndividual>

  <owl:NamedIndividual rdf:about="#CriterioInglesB2Gerente">
    <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>

```

```

<esq_dom:nombre>Nivel de ingles-B2</esq_dom:nombre>
<esq_dom:atributo rdf:resource="#Ingles"/>
<esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
<esq_dom:tipoResultado></esq_dom:tipoResultado>
<esq_dom:puntuacion>10</esq_dom:puntuacion>
<esq_dom:valor>B2</esq_dom:valor>
<esq_dom:obligatoria>false</esq_dom:obligatoria>
<esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioInglesC1Gerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-C1</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>15</esq_dom:puntuacion>
  <esq_dom:valor>C1</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioInglesC2Gerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Nivel de ingles-C2</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Ingles"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>20</esq_dom:puntuacion>
  <esq_dom:valor>C2</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioExperienciaGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Experiencia laboral</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Experiencia"/>
  <esq_dom:tipoComparacion>rango</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado>año</esq_dom:tipoResultado>
  <esq_dom:puntuacion>30</esq_dom:puntuacion>
  <esq_dom:valor>[5,60]</esq_dom:valor>
  <esq_dom:obligatoria>true</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionESOGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - ESO</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>0</esq_dom:puntuacion>
  <esq_dom:valor>ESO</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

```



```

<owl:NamedIndividual rdf:about="#CriterioTitulacionBachillerGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Bachiller</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>10</esq_dom:puntuacion>
  <esq_dom:valor>Bachiller</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionGradoMedioGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Grado Medio</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>5</esq_dom:puntuacion>
  <esq_dom:valor>Grado Medio</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionGradoSuperiorGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Grado Superior</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>15</esq_dom:puntuacion>
  <esq_dom:valor>Grado Superior</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionGradoUniversitarioGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Grado Universitario</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>30</esq_dom:puntuacion>
  <esq_dom:valor>Grado Universitario</esq_dom:valor>
  <esq_dom:obligatoria>false</esq_dom:obligatoria>
  <esq_dom:valoracion rdf:resource="#Gerente"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#CriterioTitulacionMasterGerente">
  <rdf:type rdf:resource="http://example.org/esq_dom#Criterio"/>
  <esq_dom:nombre>Titulación - Master</esq_dom:nombre>
  <esq_dom:atributo rdf:resource="#Titulacion"/>
  <esq_dom:tipoComparacion>categorica</esq_dom:tipoComparacion>
  <esq_dom:tipoResultado></esq_dom:tipoResultado>
  <esq_dom:puntuacion>60</esq_dom:puntuacion>
  <esq_dom:valor>Master</esq_dom:valor>

```



```
    <esq_dom:obligatoria>false</esq_dom:obligatoria>
    <esq_dom:valoracion rdf:resource="#Gerente"/>
  </owl:NamedIndividual>

</rdf:RDF>
```

7.2. Vista de la aplicación

En este apartado se define como se ha creado la vista de la aplicación y como se han cargado los datos de los archivos OWL. Para ello se ha utilizado la librería PyQt5 para crear la interfaz gráfica y la librería owlready2 y rdflib para cargar los archivos OWL y poder trabajar con ellos.

Enlace de descarga: [VtsValoracion.py](#)

```
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QGridLayout, QPushButton,
                             QLabel, QComboBox, QTableWidgetItem, QListWidget,
                             QTextEdit, QHeaderView, QHBoxLayout, QTableWidgetItem)

from PyQt5.QtCore import Qt
from owlready2 import *
from rdflib import Graph
from rdflib.namespace import RDF

class VtsValoracion(QWidget):
    def __init__(self):
        super().__init__()
        self.Gui()

    def Gui(self):
        # Combobox Dominio
        self.comboboxDominio = QComboBox(self)
        self.dominios = ['Becas', 'Puesto_trabajo']
        self.comboboxDominio.addItem(self.dominios)
        self.comboboxDominio.currentTextChanged.connect(self.actualizarDatosPorDominio)

        # Combobox Valoracion
        self.comboboxValoracion = QComboBox(self)
        self.comboboxValoracion.currentTextChanged.connect(self.eliminarInformacionCriterios)

        # Creación de la tabla
        self.tablaCaso = QTableWidgetItem()
        self.tablaCaso.setColumnCount(2)
        self.tablaCaso.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.tablaCaso.setHorizontalHeaderLabels(["ATRIBUTO", "VALOR"])

        # Creación de la lista de criterios
        self.listaCriterios = QListWidget()
        self.listaCriterios.currentItemChanged.connect(self.actualizarDescripcion)

        # Creación cuadros de texto
        self.editorDescripcion = QTextEdit("")
        self.editorDescripcion.setReadOnly(True)
        self.editorResultado = QTextEdit("")
        self.editorResultado.setReadOnly(True)
        self.editorJustificacion = QTextEdit("")
        self.editorJustificacion.setReadOnly(True)

        # Creación de labels
        self.labelDominio = QLabel("Dominio", self)
        self.labelValoracion = QLabel("Valoracion", self)
        self.labelCaso = QLabel("Caso", self)
        self.labelCriterios = QLabel("Criterios", self)
        self.labelDescripcion = QLabel("Descripción del criterio", self)
```

```

self.labelResultado = QLabel("Resultado final", self)
self.labelJustificacion = QLabel("Justificación de la valoración", self)

# Creación de botones
self.btnValidar = QPushButton("Validar", self)
self.btnValidar.resize(self.btnValidar.sizeHint())
self.btnSalir = QPushButton("Salir", self)
self.btnSalir.resize(self.btnSalir.sizeHint())
self.btnBorrar = QPushButton("Borrar", self)
self.btnBorrar.resize(self.btnBorrar.sizeHint())

# Asociar botones con funciones
self.btnSalir.clicked.connect(self.close)
self.btnValidar.clicked.connect(self.validarDatos)
self.btnBorrar.clicked.connect(self.borrar)

# Crear el layout
gridlayout = QGridLayout()

# Posicionar elementos columna 1
gridlayout.addWidget(self.labelDominio, 1, 1, 1, 1)
gridlayout.addWidget(self.comboboxDominio, 2, 1, 1, 5)
gridlayout.addWidget(self.labelValoracion, 3, 1, 1, 1)
gridlayout.addWidget(self.comboboxValoracion, 4, 1, 1, 5)
gridlayout.addWidget(self.labelCaso, 5, 1, 1, 1)
gridlayout.addWidget(self.tablaCaso, 6, 1, 8, 5)
gridlayout.addWidget(self.labelCriterios, 14, 1, 1, 1)
gridlayout.addWidget(self.listaCriterios, 15, 1, 5, 5)
gridlayout.addWidget(self.labelDescripcion, 20, 1, 1, 1)
gridlayout.addWidget(self.editorDescripcion, 21, 1, 8, 5)

# Posicionar elementos columna 2
gridlayout.addWidget(self.labelResultado, 1, 6, 1, 1)
gridlayout.addWidget(self.editorResultado, 2, 6, 12, 6)
gridlayout.addWidget(self.labelJustificacion, 14, 6, 1, 1)
gridlayout.addWidget(self.editorJustificacion, 15, 6, 14, 6)

# Posicionar botones
layout_botones = QHBoxLayout()
layout_botones.addStretch(1)
layout_botones.addWidget(self.btnValidar)
layout_botones.addWidget(self.btnBorrar)
layout_botones.addWidget(self.btnSalir)
layout_botones.addStretch(1)
gridlayout.addLayout(layout_botones, 30, 1, 1, 11)

#Llamamos a la funcion de actualizarDatosPorDominio ya que nos carga
#todos los datos del dominio seleccionando
self.actualizarDatosPorDominio()
self.setLayout(gridlayout)
self.setGeometry(250, 150, 1200, 850)
self.show()

def actualizarDatosPorDominio(self):
    # Cargar la ontología OWL en un nuevo mundo para que
    #no se acumulen las distintas ontologías

```

```

archivo_owl = self.comboboxDominio.currentText() + ".owl"
nuevo_mundo = World()
onto = nuevo_mundo.get_ontology('Esq_dom.owl').load()
nuevo_mundo.get_ontology(archivo_owl).load()

# Obtener las clases de la ontología
atributos = onto.Atributo
valoraciones = onto.Valoracion
criterios = onto.Criterio

#Añadimos las valoraciones
self.comboboxValoracion.clear()
for valoracion in valoraciones.instances():
    self.comboboxValoracion.addItem(valoracion.name)

#Añadimos los atributos
self.tablaCaso.clearContents()
self.tablaCaso.setRowCount(len(atributos.instances()))

for i, atributo in enumerate(atributos.instances()):
    item_atributo = QTableWidgetItem(atributo.nombre[0]) # Nombre del atributo
    item_atributo.setFlags(Qt.ItemIsSelectable | Qt.ItemIsEnabled) # Solo lectura

    # Dependiendo del tipo de atributo, asignamos el valor
    if atributo.tipo[0] == 'multiple' or atributo.tipo[0] == 'bool':
        # Si el atributo tiene opciones múltiples, usamos un ComboBox
        combo_valor = QComboBox()
        # Si el atributo es de tipo 'bool' se le asignan los valores 'True' y 'False'
        if atributo.tipo[0] == 'bool':
            atributo.posiblesValores = ['True', 'False']
        # Opciones disponibles
        combo_valor.addItems(atributo.posiblesValores)
        # Seleccionar el valor actual
        combo_valor.setCurrentText(atributo.valorInicial[0])
        # Poner el ComboBox en la celda
        self.tablaCaso.setCellWidget(i, 1, combo_valor)
    else:
        item_valor = QTableWidgetItem(atributo.valorInicial[0])
        self.tablaCaso.setItem(i, 1, item_valor) # Poner el valor en la tabla

    # Agregar el nombre del atributo
    self.tablaCaso.setItem(i, 0, item_atributo)

self.listaCriterios.clear()

#Añadimos los criterios
for criterio in criterios.instances():
    if criterio.valoracion[0].name == self.comboboxValoracion.currentText():
        self.listaCriterios.addItem(criterio.nombre[0])

def actualizarDescripcion(self, current):
    if current:
        archivo_owl = self.comboboxDominio.currentText() + ".owl"
        seleccion = self.listaCriterios.currentItem().text()
        valoracion = self.comboboxValoracion.currentText()

```

```

# Cargar ambas ontologías
g = Graph()
g.parse("Esq_dom.owl", format="xml")
g.parse(archivo_owl, format="xml")

query = f"""
PREFIX : <http://example.org/esq_dom#>
PREFIX rdf: <{RDF}>

SELECT ?nombre ?tipoComparacion ?tipoResultado ?puntuacion ?valor ?obligatoria
        ?val_id
WHERE {{
    ?criterio rdf:type :Criterio ;
                :nombre ?nombre ;
                :tipoComparacion ?tipoComparacion ;
                :tipoResultado ?tipoResultado ;
                :puntuacion ?puntuacion ;
                :valor ?valor ;
                :obligatoria ?obligatoria ;
                :valoracion ?val .

    BIND(STR(AFTER(STR(?val), "#")) AS ?val_id)

    FILTER(str(?nombre) = "{seleccion}" && str(?val_id) = "{valoracion}")
}}
"""

resultados = g.query(query)

for row in resultados:
    desc = (
        f"Nombre:\t\t{row.nombre}\n"
        f"Comparación:\t\t{row.tipoComparacion}\n"
        f"Tipo de resultado:\t\t{row.tipoResultado}\n"
        f"Puntuación:\t\t{row.puntuacion}\n"
        f"Valor:\t\t{row.valor}\n"
        f"Terminal:\t\t{row.obligatoria}"
    )
    self.editorDescripcion.setText(desc)
    break

def validarDatos(self):
    dominio = self.comboboxDominio.currentText() # Obtener el dominio seleccionado

    if dominio == "Becas":
        resultado = "Ejemplo de resultado para el dominio Becas"
        justificacion = "Ejemplo de justificación para el dominio Becas"
    elif dominio == "Puesto_trabajo":
        resultado = "Ejemplo de resultado para el dominio Puesto_trabajo"
        justificacion = "Ejemplo de justificación para el dominio Puesto_trabajo"
    else:
        resultado = "Error, dominio no reconocido"
        justificacion = "No se ha podido determinar una justificación"

    self.editorResultado.setText(resultado)
    self.editorJustificacion.setText(justificacion)

```

```
def borrar(self):
    self.editorResultado.clear()
    self.editorJustificacion.clear()

def eliminarInformacionCriterios(self):
    #Quitamos la seleccion de la lista de criterios
    self.listaCriterios.clearSelection()
    self.listaCriterios.setCurrentRow(-1)
    #Vaciamos la informacion del texto que muestra la descripcion del criterio
    self.editorDescripcion.clear()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = VtsValoracion()
    ex.show()
    sys.exit(app.exec_())
```

7.3. Video explicativo

Enlace al video: [Video explicativo](#)